

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

15. Számláló és Timer egységek, WDT

Írta: Szalay János

Lektorálta: Szabó Ádám

A beágyazott rendszerektől gyakran követeljük meg a valós idejű működést, melyet a legegyszerűbben úgy tudnánk megfogalmazni, hogy a rendszerünk a rábízott feladatot határidőre teljesíti. A végeredmény lehet sikeres vagy sikertelen, a hangsúly azon van, hogy bizonyos idő elteltével mindenképpen valamilyen eredményre jutunk.

Példaként képzeljük el, hogy az autónkkal egy nedves, csúszós úton haladunk. Bármennyire is körültekintően vezetünk, kialakulhat olyan szituáció, amely során hirtelen fékezésre kényszerülünk. Számunkra teljesen jogos elvárás lehet a biztonságos megállás, azonban járműünkben ehhez adott esetben több fedélzeti számítógép összehangolt munkájára van szükség. Ezek periodikusan információt cserélnek egymással, valamint nagy pontossággal figyelemmel kísérik az autónk mozgását és viselkedését. Adott esetben pedig beavatkoznak a vezetéstámogató és biztonsági rendszereken keresztül, hogy lehetővé tegyék a biztonságos megállást. Ehhez elengedhetetlen az egymást követő események közötti időtartam mérése vagy annak felismerése, hogy bizonyos körülmények huzamosabb időn keresztül állnak fenn. Természetesen az idő mérése és események detektálása nem valós idejű beágyazott rendszerek esetén is ugyanolyan fontos feladat lehet.

Az említett határidő tartásához rendkívül precíz időmérésre van szükségünk, amit a mikrokontrollerek gyártói külön erre a célra tervezett perifériákon - az úgynevezett számlálón (Counter) és időzítőn (Timer) - keresztül biztosítanak számunkra.

Az időzítésen kívül még sok egyéb hasznos feladat ellátásra használhatjuk ezeket a modulokat, amelyeket később részletesen ismertetünk.

Foglalkozunk még a watchdog modullal is, amely ugyan különálló perifériaként található meg általában egy mikrovezérlőben, de ugyanúgy egy időzítőn keresztül véd minket minden programozó egyik rémálmától, a szoftver megakadásától.

A SZÁMLÁLÓ ÉS IDŐZÍTŐ MODULOK ÜZEMMÓDJAI

A számláló (Counter) periféria segítségével a mikrovezérlőn kívül bekövetkező események számosságát határozhatjuk meg.

Az időzítő (Timer) meghatározott időközönként (órajelre) növeli vagy csökkenti egy regiszter értékét, ezáltal tartja számon az idő múlását.

A mikrokontrollerek gyártói ezt a két funkcionalitást gyakran egy perifériába integrálják, amelyre ebben a tananyagrészen időzítő és számláló (Timer/Counter) modulként hivatkozunk.

Általában több hasonló periféria is megtalálható egy mikrokontrollerben, azonban fontosnak tartom megjegyezni, hogy ezek gyakran különböznek egymástól bizonyos funkciók tekintetében. A

tananyagban használt mikrovezérlő gyártója ezt három kategóriába sorolva tünteti fel: haladó/fejlett (advanced), általános célú (general purpose) és alap/egyszerű (basic). Itt nem állhatunk meg persze, ugyanis a felsorolt kategóriákon belül is találunk eltéréseket az egyes modulok között, ezért a legfontosabb tulajdonságokat az alábbi táblázatban összefoglaltam. Azonban ezek mellett a perifériák még sok egyéb funkcióval rendelkezhetnek, amelyek befolyásolják a besorolásukat.

Kategória	Periféria neve	Számláló felbontása	Számlálás iránya	Dedikált lábak száma
Haladó/fejlett	TIM1, TIM8	16 bit	Fel, le, fel/le	4
Általános célú	TIM2, TIM5	32 bit	Fel, le, fel/le	4
	TIM3, TIM4	16 bit	Fel, le, fel/le	4
	TIM9	16 bit	Fel	2
	TIM10, TIM11	16 bit	Fel	1
	TIM12	16 bit	Fel	2
	TIM13, TIM14	16 bit	Fel	1
Alap/egyszerű	TIM6, TIM7	16 bit	Fel	0

1. táblázat: A Timer/Counter modulok főbb paraméterei

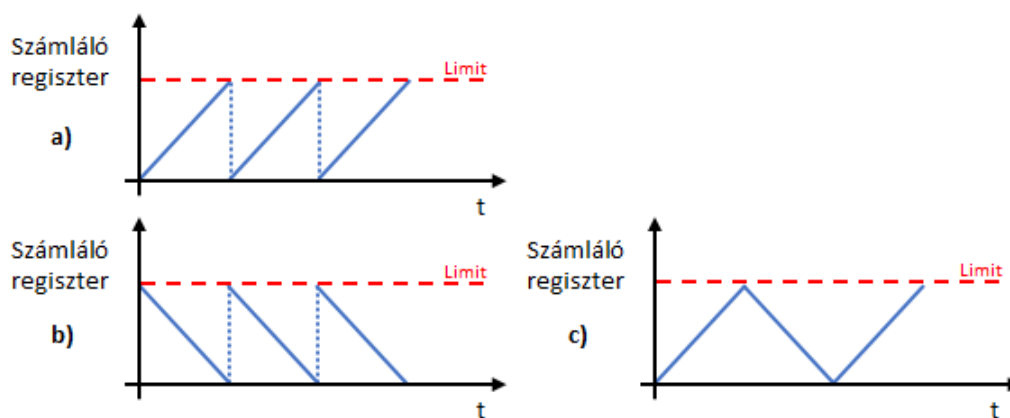
Mielőtt rátérnénk a számláló és időzítő modulok egyes üzemmódjaira, érdemes áttekinteni a táblázatban szereplő főbb paramétereket, hiszen ezek általános ismerete nélkülözhetetlen az egyes üzemmódok konfigurálásához.

A számláló felbontása a periféria számláló regiszterének méretére utal, amely egyértelműen meghatározza a beállítható maximális időzítés hosszát. Minél több biten képes számolni a modulunk, annál hosszabb időzítéseket állíthatunk be (ugyanakkora órajel frekvencia esetén), emellett adott frekvencia/időzítés esetén pontosabb működést is biztosít. Gondoljunk csak bele, egy óra esetén 1 év (számláló regiszter mérete) alatt 1 perc időzítési hiba nem tűnik jelentősnek, ehhez képest 1 nap alatt ugyanekkora eltérés már számottevő.

Fontos megjegyezni, hogy ugyanazt a számláló regisztert használja a Timer és a Counter periféria is.

A számlálás irányával azt állíthatjuk be, hogy a számláló regiszterünk értéke (amely csak pozitív egész szám lehet) növekedjen, csökkenjen, vagy adott esetben ezeket váltogatva felfelé, majd lefelé számoljon. Felfelé számlálás esetén a számláló regiszterünk 0 értékről indul és egészen addig növekszik az értéke, amíg el nem éri a számlálási limitet, melyet egy dedikált regiszterben állítunk be, értéke legfeljebb a számláló regiszter maximális értékével lehet azonos. Lefelé számlálás esetén a beállított limitről indul a számláló regiszter értéke és addig tart a számlálás, míg el nem éri a 0-át. A

fel/le számlálás pedig az előző két mód hibridjeként értelmezhető: 0-tól kezdve elszámol a modul a beállított limit értékig, majd a limitértékről 0-ig. A számlálási irányokat ismertetik az alábbi ábrák. Az **a)** ábrán egy felfelé, a **b)** -n egy lefelé, míg a **c)** -n egy felfelé/lefelé számláló modul számláló regiszterének változása látható.



1. ábra: lehetséges számlálási irányok

Kitekintés:

Ameddig csak egyszerű időzítéseket szeretnénk megvalósítani, nyugodtan választhatunk a felfelé vagy lefelé számlálási irányok közül. Villanymotorok szabályozása esetén azonban előszeretettel alkalmazzák a felfelé/lefelé számláló módot, ugyanis ezzel kevesebb harmonikusból álló jelet lehet előállítani. Ennek köszönhetően pedig kisebb teljesítmény veszteség jelentkezik a motorvezérlő áramkörében.

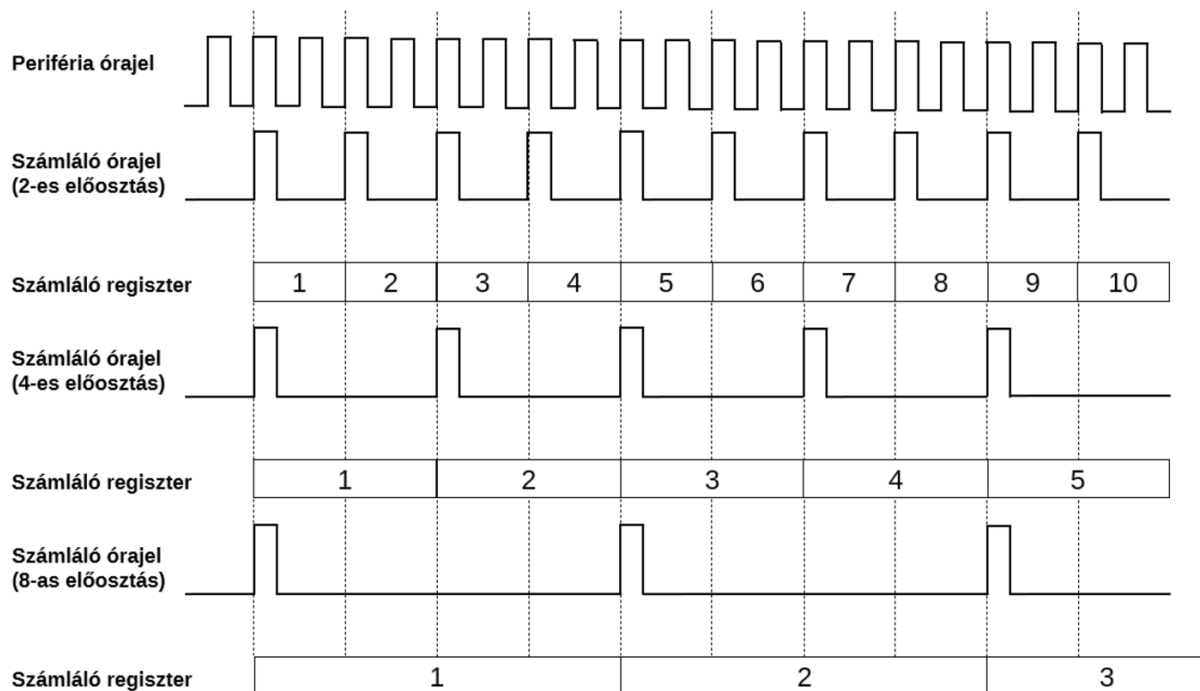
A számláló és időzítő perifériák rendelkezhetnek dedikált kimenetekkel és bemenetekkel, amelyek egyfajta interfészt biztosítanak a modul és a külvilág között. Külső események hatására változtathatjuk a számláló regiszter értékét, vagy az impulzusok hosszát mérhetjük meg. A kimenetekkel pedig a beállított időzítések lejártát jelezhetjük jelszintváloztatással, vagy adott esetben villanymotorok hajtásához állíthatunk elő pontos vezérlőjelet. Beágyazott rendszereink tervezésénél mindig ügyelnünk kell arra, hogy mely mikrovezérlő lábak mely számláló és időzítő modulhoz tartoznak, azaz a tervezett funkció ellátásához a megfelelő képességekkel rendelkező perifériát válasszuk.

IDŐZÍTŐ FUNKCIÓ

A számláló felbontását már az előző táblázatban beláttuk, a periféria ezen tulajdonsága komoly hatással van az időzítés maximális hosszára, vagyis minél több biten tároljuk a számláló regiszter értékét, annál nagyobb értékig tudunk elszámolni.

Emellett általában vannak még további lehetőségeink az időzítéseink hosszának beállítására, azonban ezek mikrovezérlő függőek. Egyes típusoknál beállíthatjuk a periféria által használt órajel forrását is, ezzel alapjaiban meghatározhatjuk a számlálás sebességét. Az általunk használt STM32 esetében például külső órajelforrást köthetünk a dedikált lábakra. Szinte minden mikrokontroller esetében lehetőségünk van a számláló perifériához előosztás (prescaler) értéket megadni, ami a modul órajelét

még tovább csökkenti. Erre azért van szükség, mert magas órajellel azonos felbontás mellett sokkal rövidebb idő alatt csordul túl a számláló.



2. ábra: Alacsonyabb frekvenciájú órajelek előállítás

Egy időzítés hosszát ezek mellett alapvetően befolyásolja a már korábban is említett limitérték, amely meghatározza, hogy meddig számoltatunk el a mikrovezérlővel.

Általános esetben, ha a lehető leghosszabb időzítést szeretnénk megvalósítani egy számlálóval, azt a következő módokon tehetjük meg:

- a periféria órajelét a lehető legkisebb értékre,
- az előosztást a lehető legmagasabbra,
- a számlálás határát a számláló regiszter maximális értékére állítjuk.

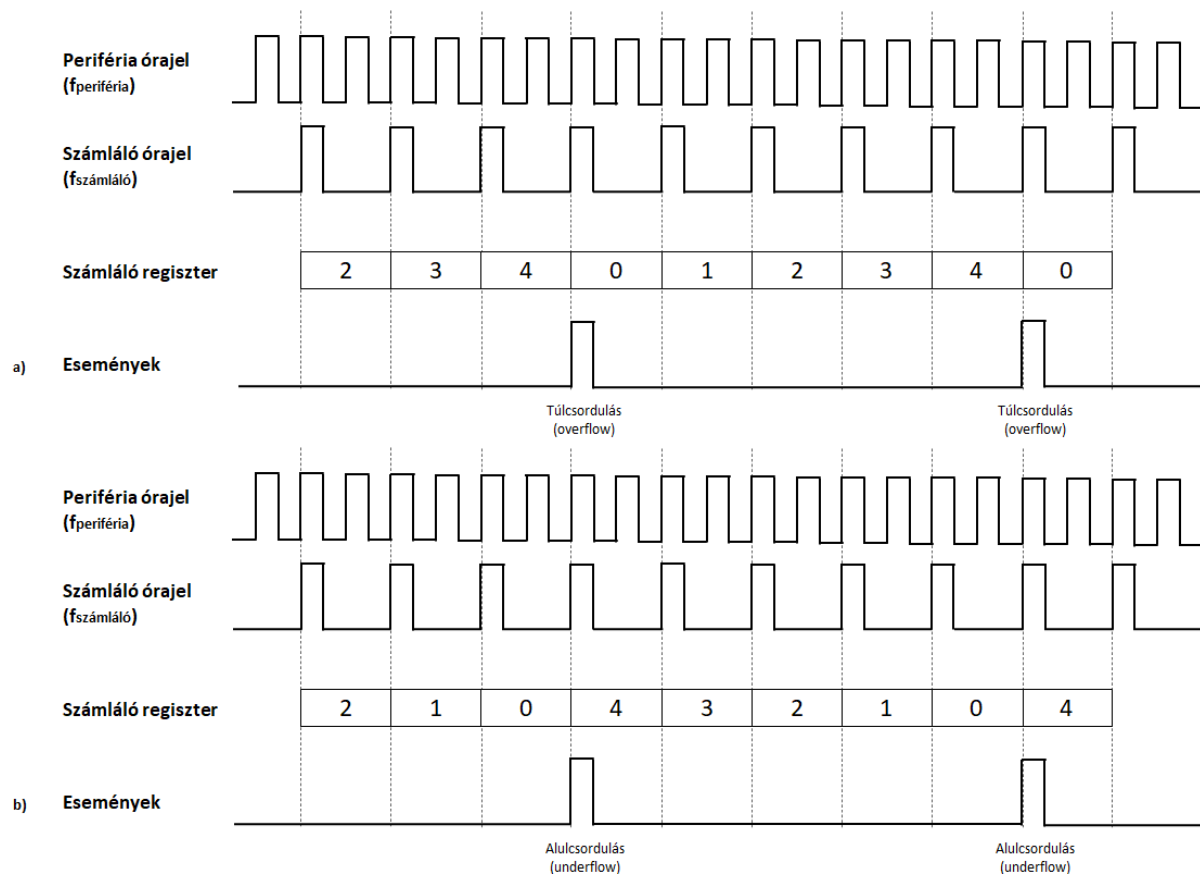
A fentebb leírtakat a következő képlettel foglalhatjuk össze:

$$T [s] = \frac{\text{Előosztás} \times \text{Limit}}{f_{\text{periféria}} [Hz]},$$

ahol:

- **T** az időzítés hosszát jelöli,
- **$f_{\text{periféria}}$** a számláló és időzítő periféria órajele,
- **Előosztás** az előosztó (prescaler) regiszter értéke,
- **Limit** a számlálás határát beállító regiszter értéke.

A fentebb leírtak egyszerűbb megértése érdekében vizsgáljuk meg a következő ábrákat, amelyek a számláló regiszter változását mutatják be, 2-es leosztásra konfigurálva. Az **a)** ábrán azt láthatjuk, hogy felfelé számol a modul 4-ig, míg a **b)**-n lefelé számol 4-ről. Ezeken azt is megfigyelhetjük, hogy mikor generál a periféria eseményeket. Amennyiben ezeket az eseményeket engedélyezzük, akkor a bekövetkezésükkor meghívódik a periféria megszakításkezelő rutinja, azaz értesülünk róla, hogy letelt az általunk beállított időzítés.



3. ábra: Túlcsordulás és alulcsordulás események bekövetkezése

A tananyagban használt mikrokontroller időzítő és számláló moduljait bemutató táblázatban láthatjuk, hogy a perifériák többsége csak a felfelé számlálás üzemmódot támogatja. Ezeknél a moduloknál a számláló regiszter értékét a periféria csak növelni tudja, ebből következőleg csak túlcsordulás esemény generálódik, amikor a számláló regiszter értéke elérte a beállított limitet.

A bemutatott számláló funkciót rengeteg célra alkalmazhatjuk a gyakorlatban. Segítségünkre lehet periodikus események ütemezésére, amelyre talán a legegyszerűbb példa lehet a LED-ek villogtatása. Használhatjuk emellett egyszerű, rögzített idő kiváráására is.

A beágyazott rendszereknél gyakran előfordul, hogy szoftverünkkel alkalmazkodnunk kell a hardverrel járó megkötésekhez. Képzeljük el, hogy mikrovezérlőnkkel egy WiFi modulon keresztül szeretnénk kommunikálni a külvilággal, azonban energiatakarékossági szempontokat figyelembe véve a kommunikációs eszközt kikapcsoljuk, ha nincs rá szükségünk. Miután bekapcsoljuk a WiFi modult, valamennyi időt várunk kell, hogy a tápfeszültsége felfusson a megfelelő szintre és kommunikációt kezdeményezzünk vele. Az ilyen és ehhez hasonló időzítések kiváráására kiválóan alkalmasak a számláló és időzítő modulok.

További hasznos alkalmazási terület a kommunikációs időtűllépés felügyelete. Egyfelől védhetjük az általunk kezdeményezett üzenetváltásokat, ugyanis a küldendő adat mennyiségéből és az adatsebességéből meghatározható, hogy mennyi idő alatt kell végeznünk az üzenetünk elküldésével. Másfelől időtűllépést határozhatunk meg a mikrovezérlőnkkel kommunikáló eszköz válaszának beérkezésére is. Ezek a védelmek különösen hasznosak lehetnek számunkra, amikor több modul kooperációjával valósítjuk meg a kommunikációt, például UART és DMA összekötésével.

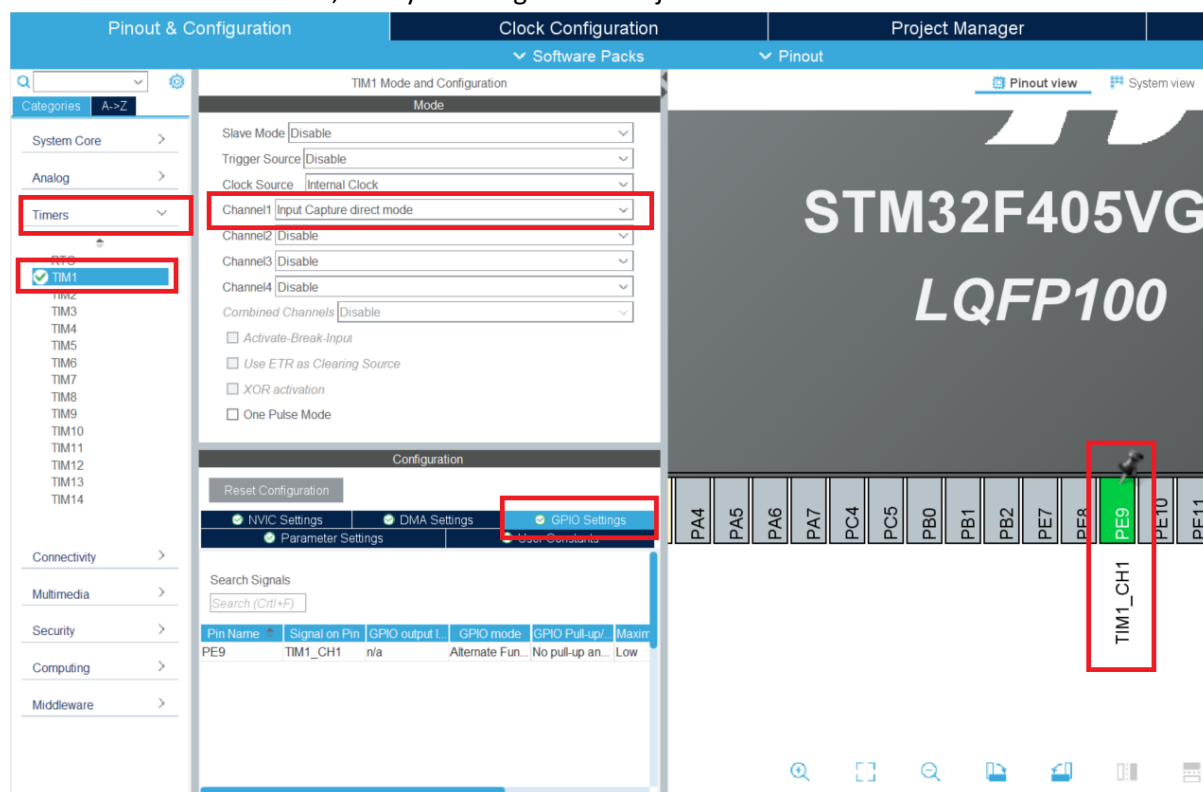
BEMENET DETEKTÁLÓ ÜZEMMÓD

A bemenet detektáló (Input Capture) üzemmód, ahogy a neve is árulkodik róla, lehetővé teszi a számláló és időzítő modulok dedikált bemenetein bekövetkező események detektálását és az ezek között eltelt idő mérését.

A beágyazott rendszerekbe fejlesztett funkciók gyakran határozott jelszint változással jelzik a mikrovezérlő felé bizonyos események bekövetkezését, amelyek feldolgozását már a szoftver végzi. Amennyiben a rendszerünk számára fontos információtartalommal bír az adott események időtartama vagy ismétlődési gyakorisága, akkor érdemes ezt egy analóg kapcsolás segítségével jelszint változássá konvertálni és rákötni a mikrokontrollerünk egyik alkalmas számláló és időzítő moduljának dedikált bemenetére.

Az üzemmód működését legkönnyebben úgy érthetjük meg, ha sorra vesszük, hogy mi történik a perifériával egy esemény detektálása esetén. Előtte azonban tekintsük át a leggyakrabban előforduló beállításokat. Az egyes lépéseknél az STM32CubeIDE fejlesztői környezetben szemléltetem a lehetőségeket, a TIM1 periféria 1-es csatornájához tartozó (Channel 1) dedikált lábának használatával.

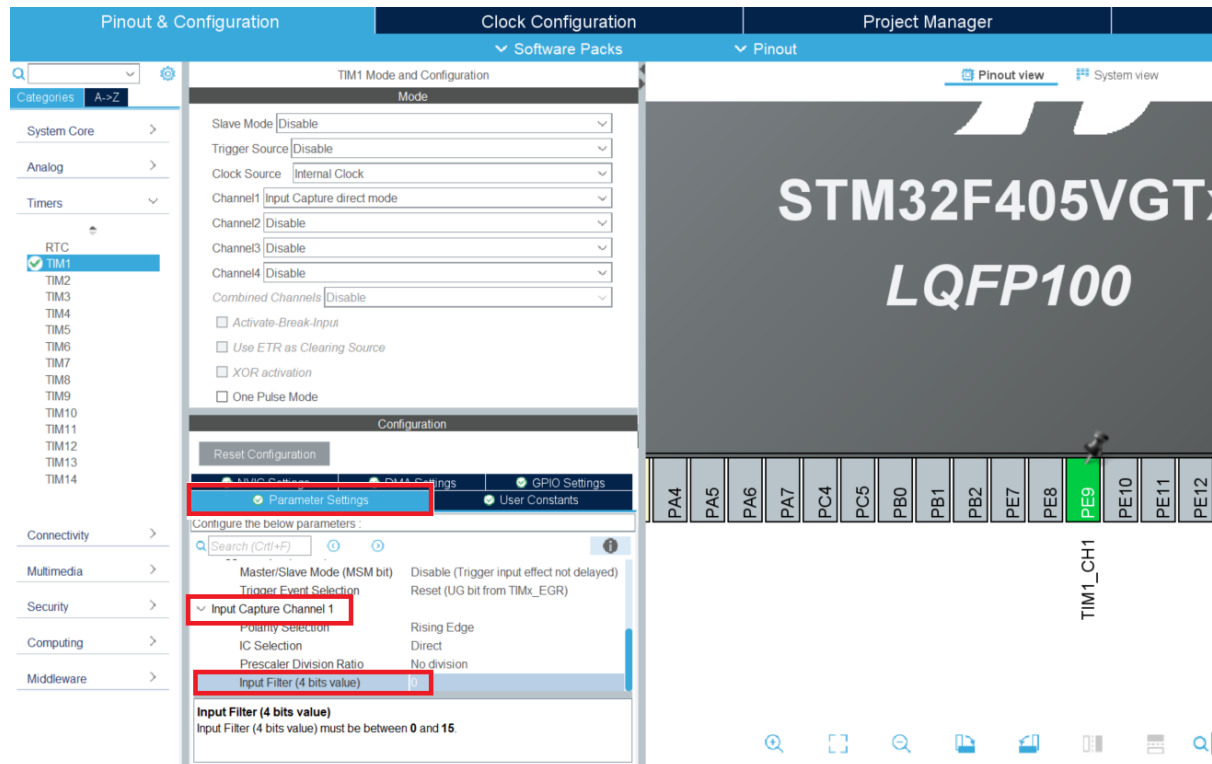
- 1) Először is engedélyezni kell a modulunkban a megfelelő Capture/Compare csatornát, vagyis azt a dedikált lábat, amelyre a vizsgálni kívánt jelet kötöttük.



4. ábra: Capture/Compare csatorna kiválasztása

Az órajel forrását (Clock Source) állítsuk belsőre (Internal Clock), a Channel 1-nél pedig válasszuk ki az Input Capture direct mode-ot. A mikrovezérlő ábráján a környezet be is jelöli a PE9 lábát, ezen keresztül tudjuk majd érzékelni a külvilág eseményeit.

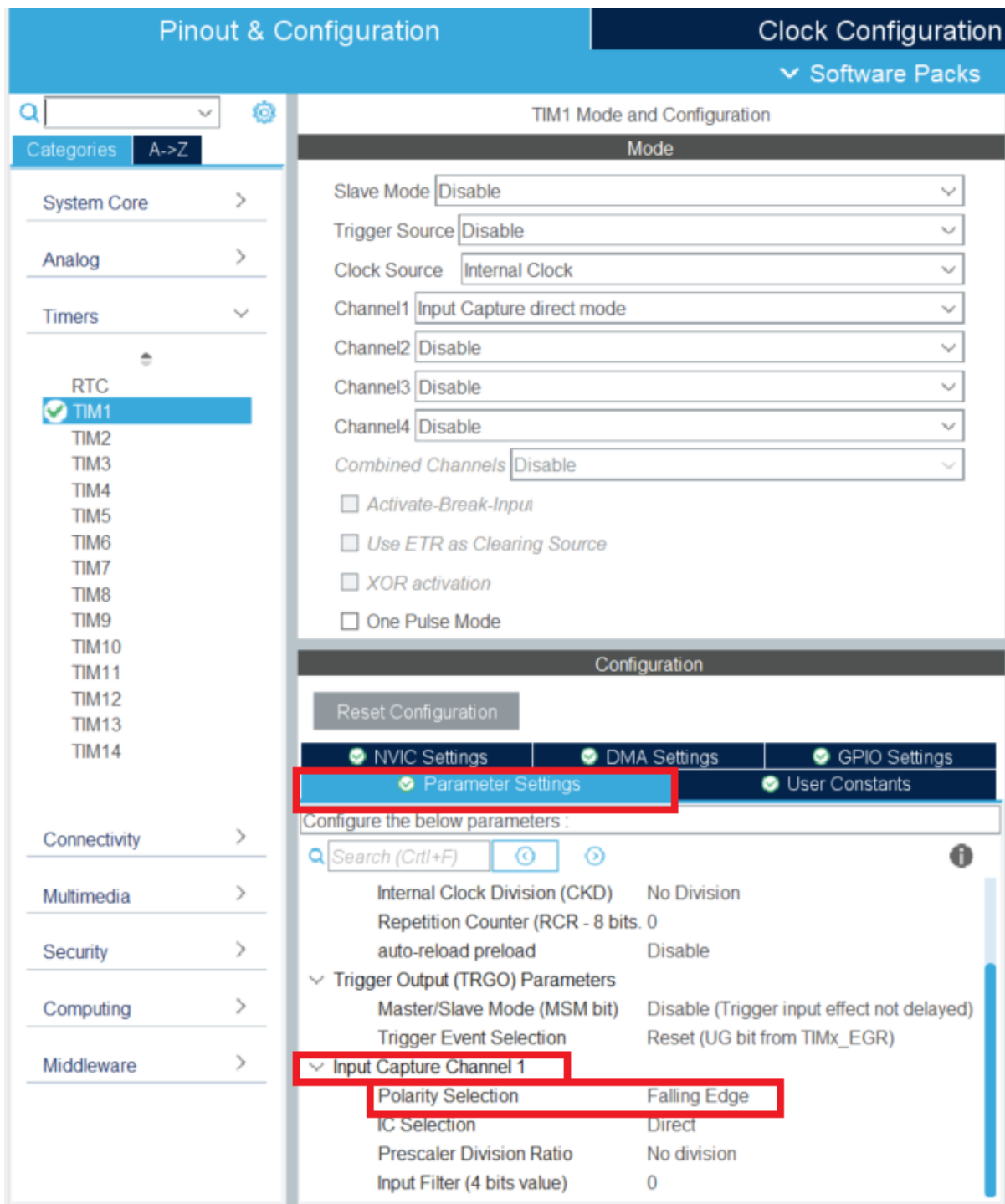
- Érdemes lehet beállítani a bemenetre szűrőt (Input Filter), melyet a mikrovezérlők többsége támogat. Ez azért hasznos, mert segítségével megsűrűzhetők a zajos vagy esetlegesen pergő jelek, így elkerülhetjük a többszörös detektálást. A pergő jelekről bővebben az I/O portok, 7. tananyagrészen lehet olvasni.



5. ábra: Bemeneti szűrő beállítása

A szűrő beállítását a Parameter Settings-en belül tehetjük meg, az Input Filter mezőbe történő szám beírásával. Ezt a paramétert 5-re választottam, így a jelszint megváltozását követően 5 azonos minta szükséges az új logikai jelszintből ahhoz, hogy a periféria is detektálja a változást.

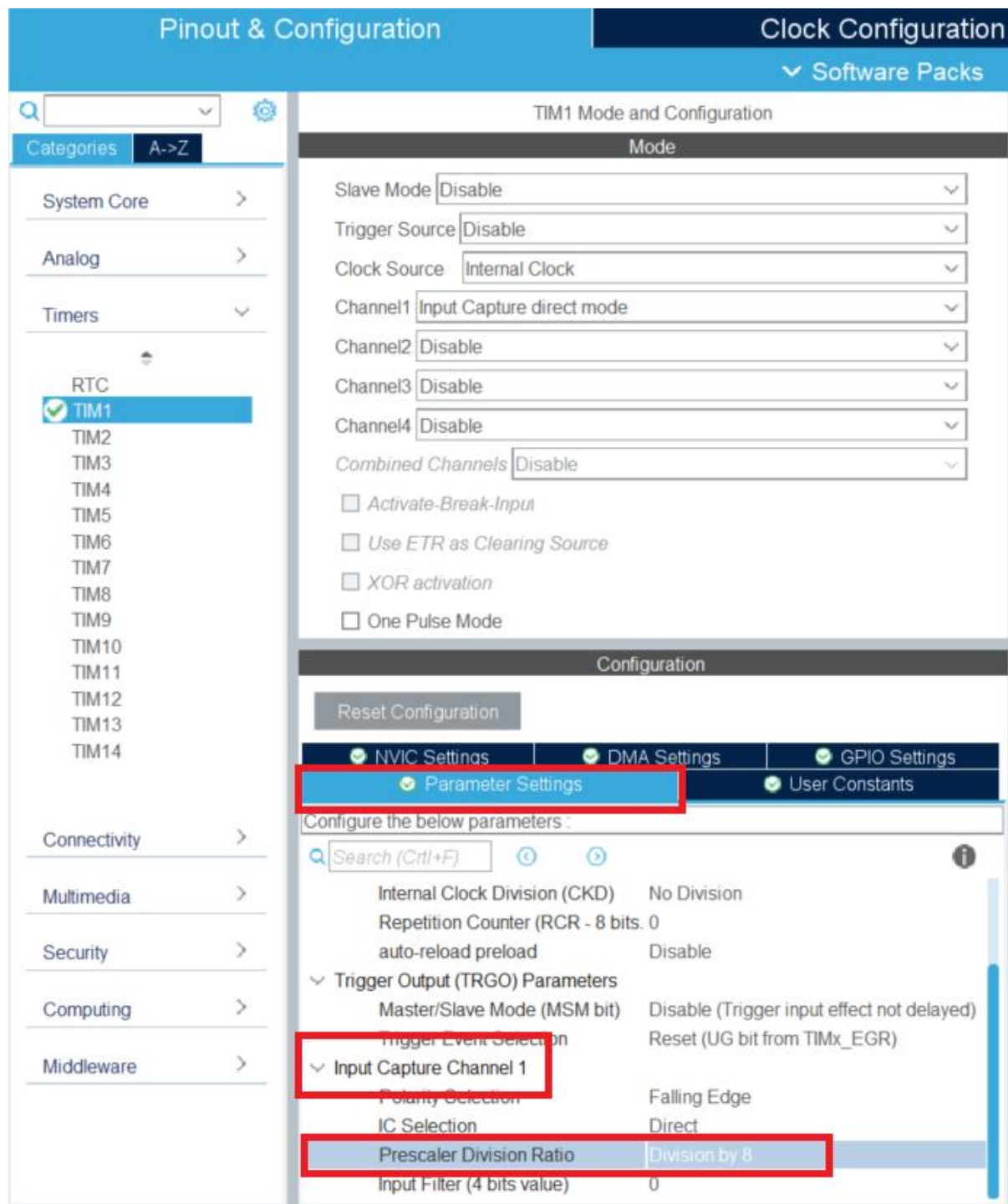
- Be kell állítani, hogy milyen irányú jelszintváltozást szeretnénk detektálni. Erre a legtöbb esetben három lehetőségünk is van: beállíthatunk csak felfutó él, vagy csak lefutó él, illetve felfutó és lefutó él detektálására is esemény generálást. A példa kedvéért lefutó élre állítottam a detekciót, amit a Polarity Selection-nél lehet kiválasztani.



6. ábra: Detekció beállítása a jelszint változás irányára vonatkozóan

- 4) Általában lehetőségünk van előosztást alkalmazni, amivel a számláló üzemmódnál is találkoztunk. Ebben az esetben az előosztás azt jelenti, hogy hányadik jelszint változásra szeretnénk eseményt generálni. Ennek beállítását a Prescaler Division Ratio mezőnél végezhetjük el. A példában ezt 4-re (Division by 4) állítottam, így minden negyedik él esetén

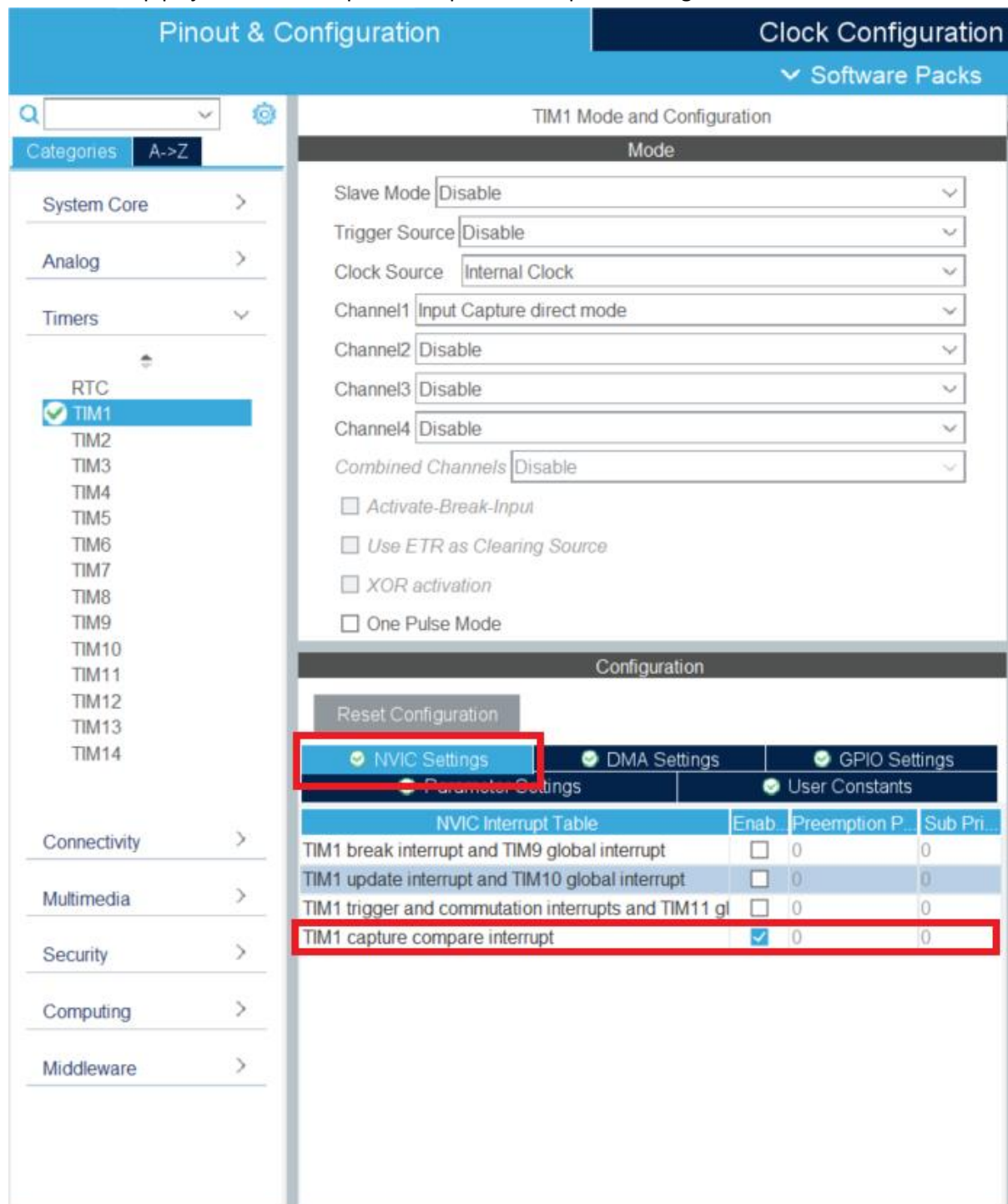
jelez majd a periféria detekciót. Erről bővebben a reference manual ¹ 574. oldalán lehet olvasni.



7. ábra: Előosztás beállítása

¹ https://www.st.com/resource/en/reference_manual/rm0090-stm32f405415-stm32f407417-stm32f427437-and-stm32f429439-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

- 5) Végül pedig engedélyezni kell a modul megszakítását a megfelelő Capture/Compare csatornákra, hogy értesüljünk a bekövetkezett eseményekről. Ehhez az NVIC Settings-en belül pipáljuk ki a TIM1 capture compare interrupt lehetőségét.



8. ábra: A megszakítás engedélyezése

Az események detektálásához el kell indítanunk a számláló és időzítő modult. Amint bekövetkezett az általunk detektálni kívánt esemény a mikrovezérlő dedikált lábán, a mikrokontroller kimentí a háttérben folyamatosan futó számláló regiszter aktuális értékét a Capture/Compare csatornához tartozó regiszterbe. Ezt követően lefut a megszakításkezelő rutinunk, amelyben kiolvashatjuk a

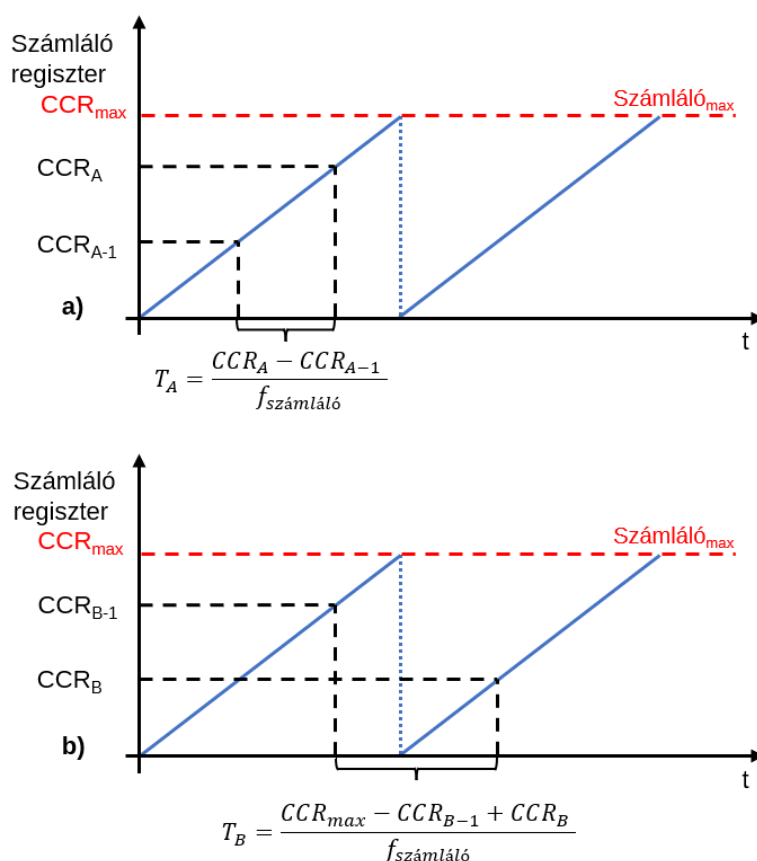
Capture/Compare regiszter értékét, azaz eltárolhatjuk az esemény bekövetkezésének pontos idejét. Az egymást követő események időbélyegeiből pedig egyszerűen kiszámolhatjuk a közöttük eltelt időt az alábbi összefüggés segítségével:

$$T [s] = \frac{CCR_N - CCR_{N-1}}{f_{számláló} [Hz]},$$

ahol:

- T a két esemény között eltelt időt jelöli,
- CCR_N a Capture/Compare regiszter aktuális értéke,
- CCR_{N-1} a Capture/Compare regiszter korábbi értéke,
- $f_{számláló}$ az a frekvencia, amellyel a perifériánk számláló regisztere növeli az értékét.

Figyelnünk kell azonban arra az esetre, amikor a két esemény bekövetkezése között pont átfordul a számláló. Erre mutat példát a következő ábra.



9. ábra: Az események között eltelt idő meghatározása túlsordulás nélkül (a) és túlsordulás esetén (b)

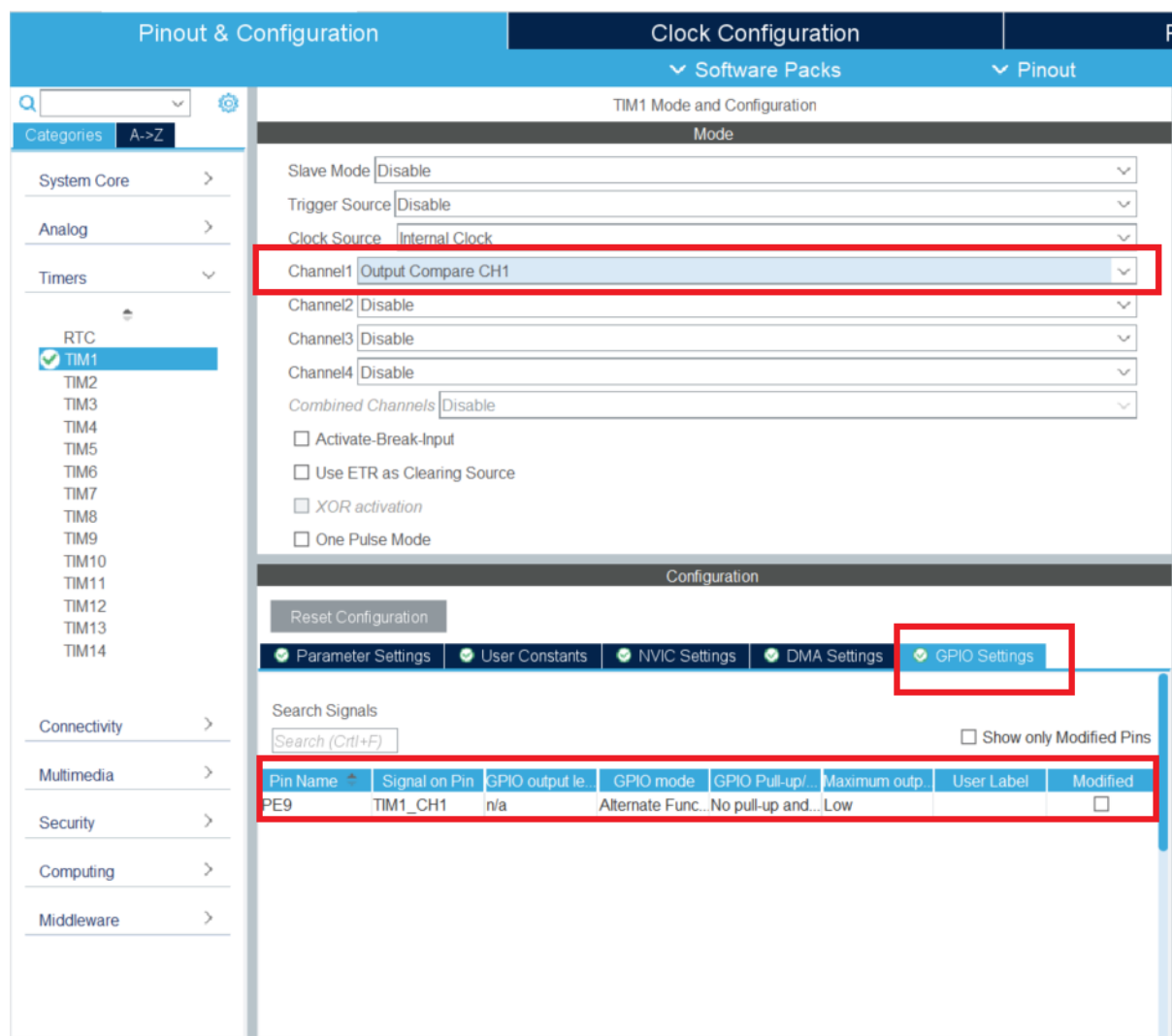
IDŐZÍTETT KIMENET ÁLLÍTÁS

Az Időzített kimenet állítás, azaz az Output Compare üzemmód lehetővé teszi a számláló és időzítő modulok dedikált lábain történő beavatkozást. Erre a célra ugyanazokat a lábakat használhatjuk, mint a fentebb megismert Input Capture mód esetén, csak ebben az esetben kimenetként funkcionálnak ezek a lábak.

Beágyazott rendszereink működésébe gyakran logikai jelek alacsony vagy magas szintbe állításával avatkozhatunk be, azaz a mikrokontroller lábát a rendszerünk adott funkcióját megvalósító áramköri kapcsolás vezérlésére használjuk. Az Output Compare mód segítségével nagyon pontos időzítések megvalósítása révén gyakorolhatunk hatást rendszerünk viselkedésére.

Az Input Capture módhoz hasonlóan első körben tekintsük át a leggyakoribb beállításokat, mielőtt rátérnénk a működési mechanizmus részletezésére. Az egyes lépéseknél a fejlesztői környezetben szemléltetem a beállításokat a TIM1 periféria Channel 1 dedikált lábának használatával.

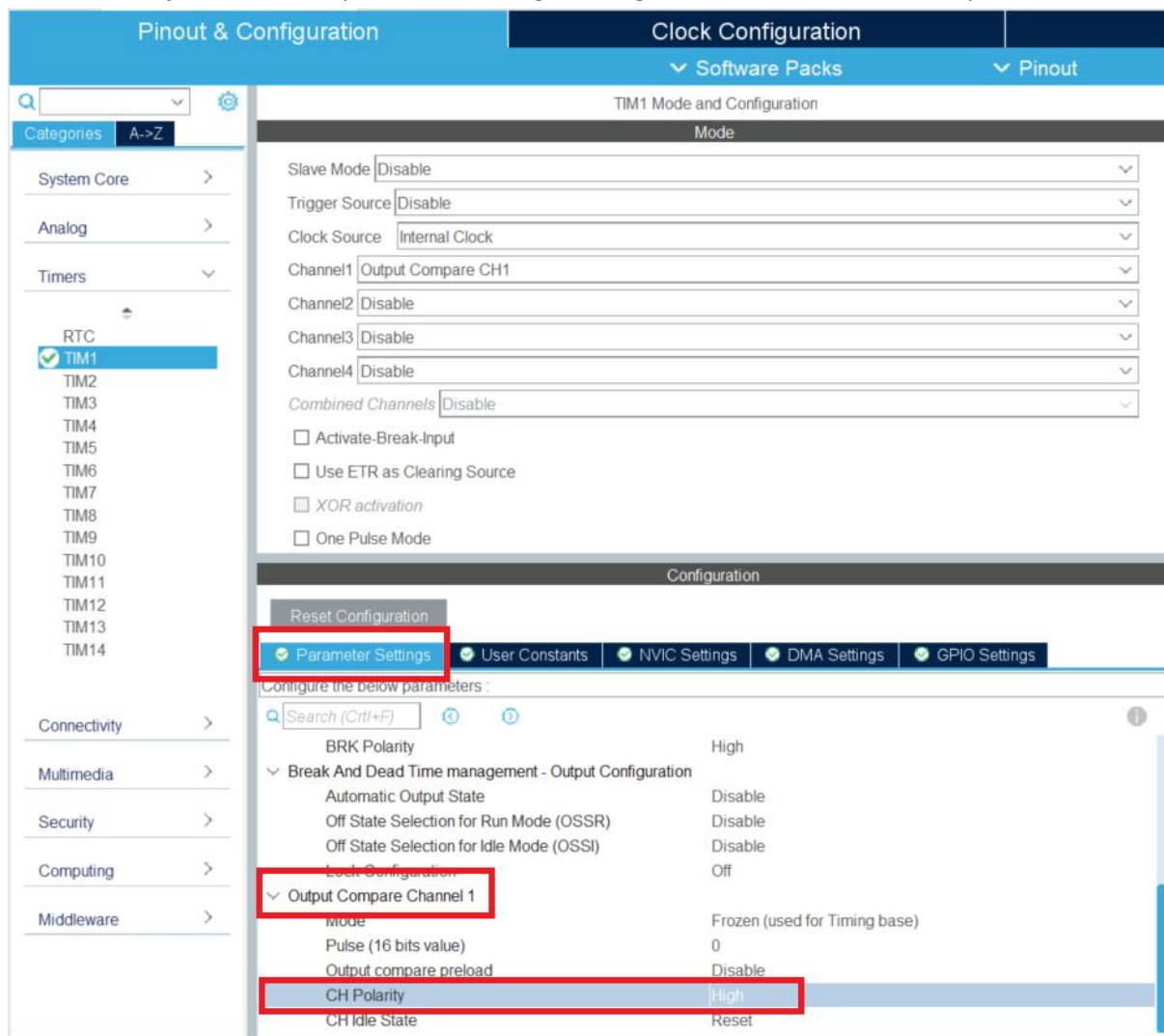
- 1) Először is engedélyezzük és állítsuk kimenetre a modulunkban a megfelelő Output Compare csatornát, vagyis azt a dedikált lábat, amelyen keresztül beavatkozni kívánunk a rendszerünk működésébe.



10. ábra: Output Compare csatorna kiválasztása

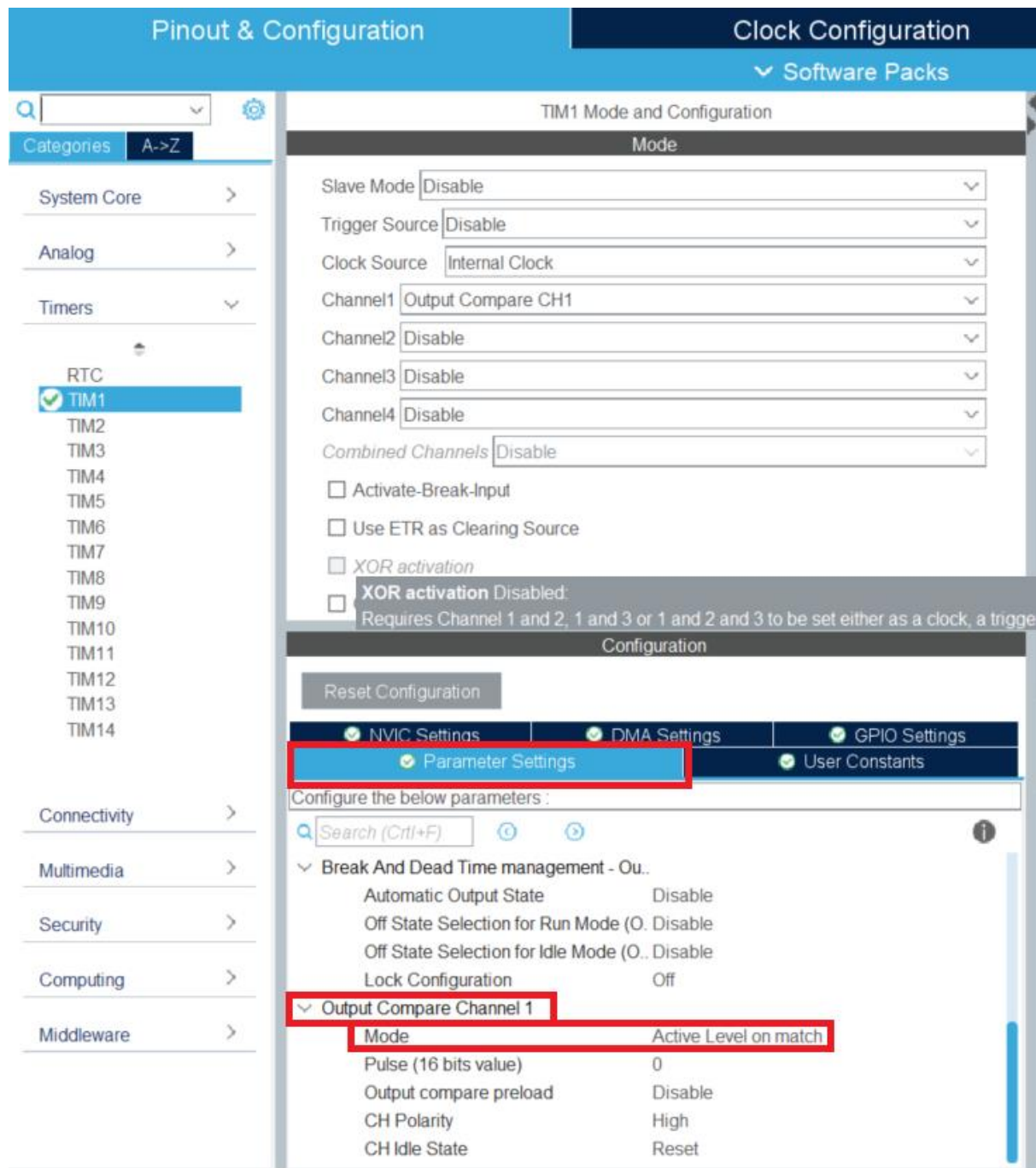
A Clock Source-ot állítsuk Internal Clock-ra, a Channel 1-nél pedig válasszuk ki az Output Compare CH1 lehetőséget. A mikrovezérlő ábráján a környezet be is jelöli a PE9 lábat, ezen keresztül tudunk majd hatást gyakorolni a rendszerünkre.

- 2) Be kell állítanunk a dedikált láb polaritását, azaz, hogy az alacsony vagy a magas jelszintet tekintjük aktívnak. A példában ezt magasra (High) választottam a CH Polarity mezőnél.



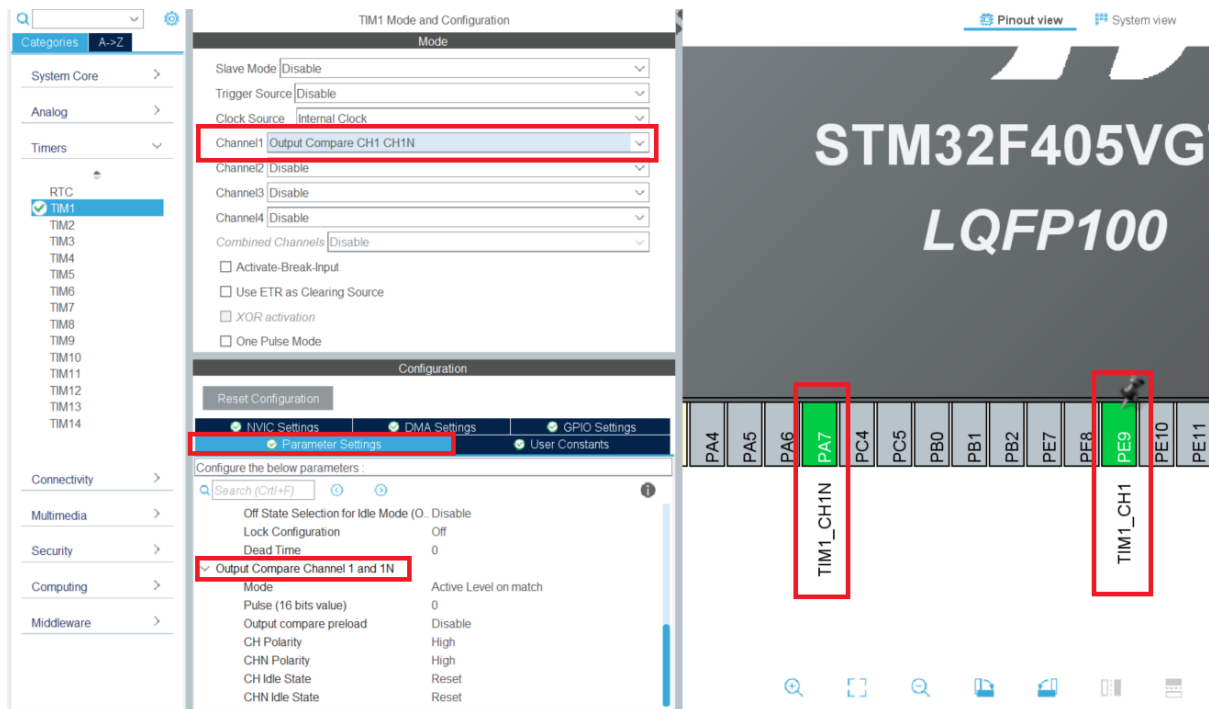
11. ábra: A dedikált láb polaritásának beállítása

- 3) Ki kell választanunk, hogy mi történjen az időzítés lejártakor. Erre általában több lehetőségünk is van: beállíthatjuk az aktív vagy az inaktív jelszintet a kimenetre. Gyakran lehetőségünk van az aktuális kimeneti jelszint ellenkezőjét is beállítani (toggle funkció). A lehetőségek közül a Mode mezőnél választhatunk. Jelen esetben az Active Level on match beállítás hatására az aktív, azaz a logikai magas jelszint kerül majd a kimenetre az időzítés lejártakor.



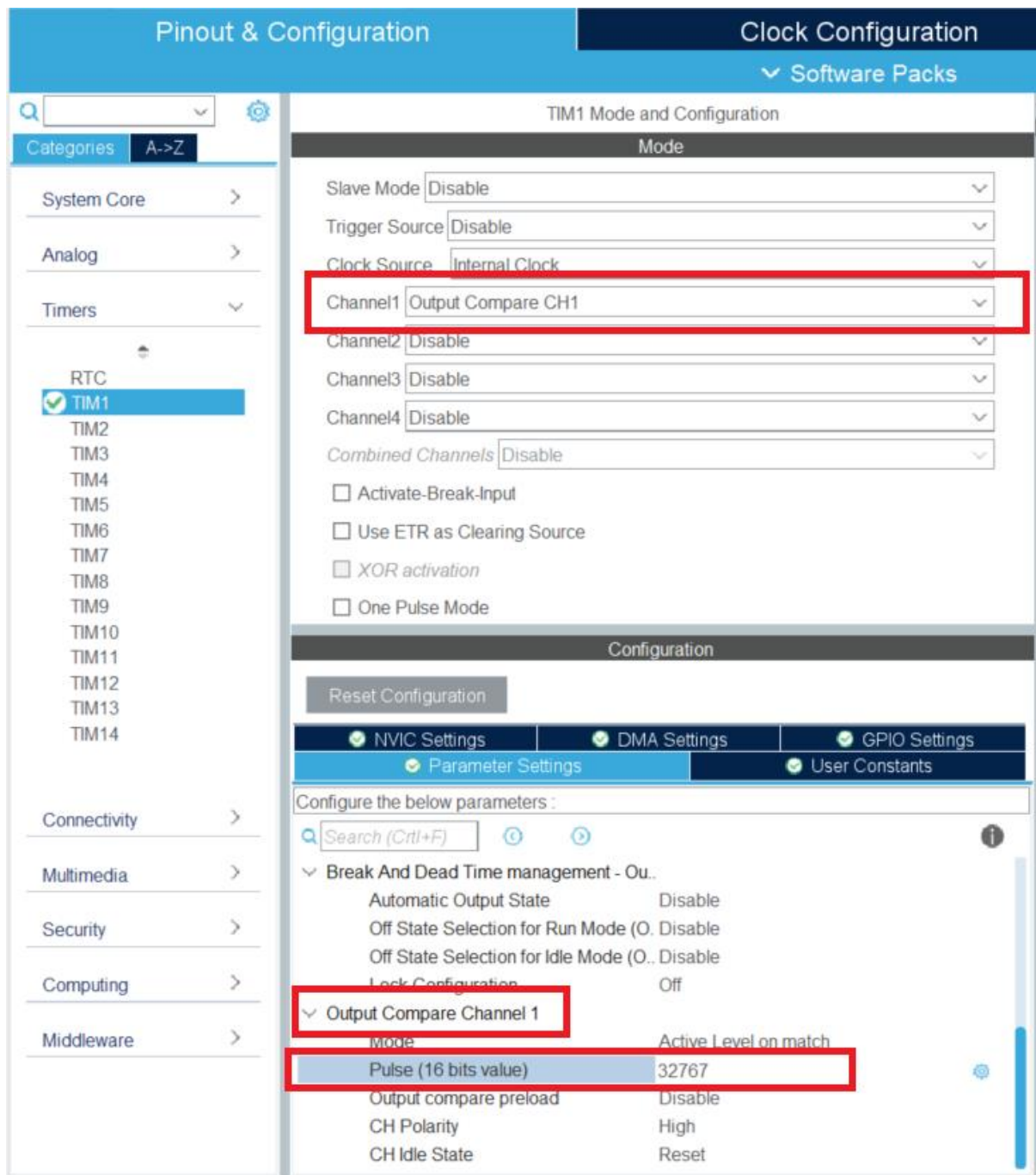
12. ábra: Az időzítés lejártá esetén alkalmazandó beavatkozás kiválasztása

- 4) Érdemes megjegyezni, hogy gyakran két mikrovezérlő lábat is vezérelhetünk egy Output Compare csatornáról, ezek közül az egyiket a kiegészítő (complementary) jelzővel szokták a gyártók ellátni. Ebben az esetben lehetőségünk van mindkét, vagy csak az egyik láb használatára, gyakran az aktív polaritásukat is külön-külön megadhatjuk. A példánknál maradvára erre akkor van lehetőségünk, ha a "Channel 1" mezőnél az "Output Compare CH1" lehetőséget választjuk a legelső lépésben.



13. ábra: Kiegészítő (complementary) láb beállításai

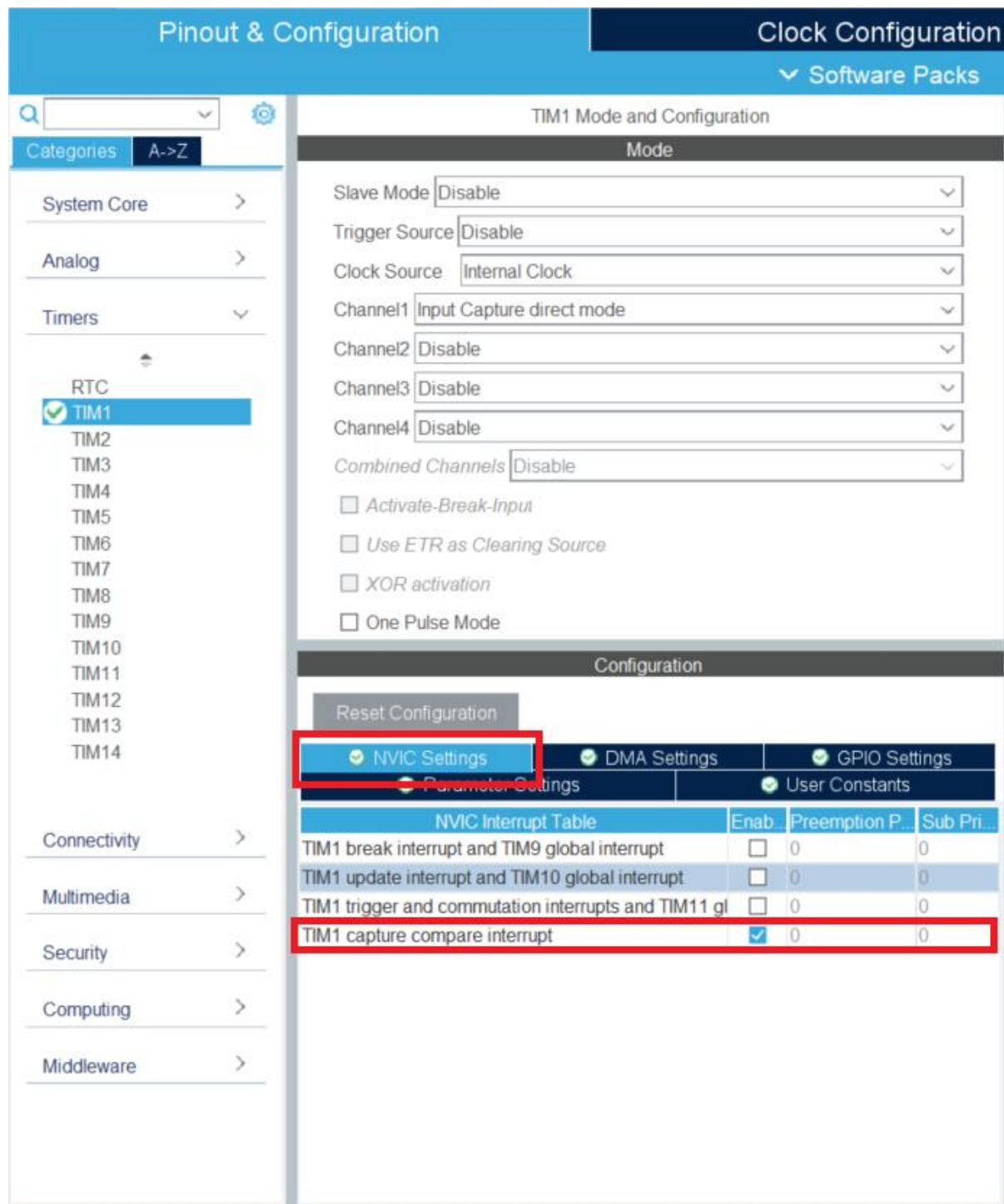
- 5) El kell végeznünk az Output Compare csatornákhhoz tartozó Capture/Compare regiszter beállítását. A mikrovezérlő lábán a kívánt jelszint beállítása ugyanis akkor történik meg, amikor a számláló regiszter értéke megegyezik a Capture/Compare regiszter értékével, azaz letelt a beállított időzítésünk.



14. ábra: Az időzítés hosszának meghatározása

A példa kedvéért használjuk ki a teljes számlálási tartományt, a dedikált lábon való beavatkozás pedig történjen az időzítés felénél. Ennek megfelelően a számlálási limit 65 535, a Pulse mezőhöz pedig 32 767 érték került.

- 6) Végül pedig engedélyezni kell a modul megszakítását a megfelelő Capture/Compare csatornákra, hogy értesüljünk az időzítés lejártának pillanatáról. Amennyiben rögtön egy újabb időzítést szeretnénk indítani, akkor ezt megtehetjük a megszakításkezelő rutinban. Ehhez az NVIC Settings-en belül pipáljuk ki a TIM1 capture compare interrupt lehetőségét.

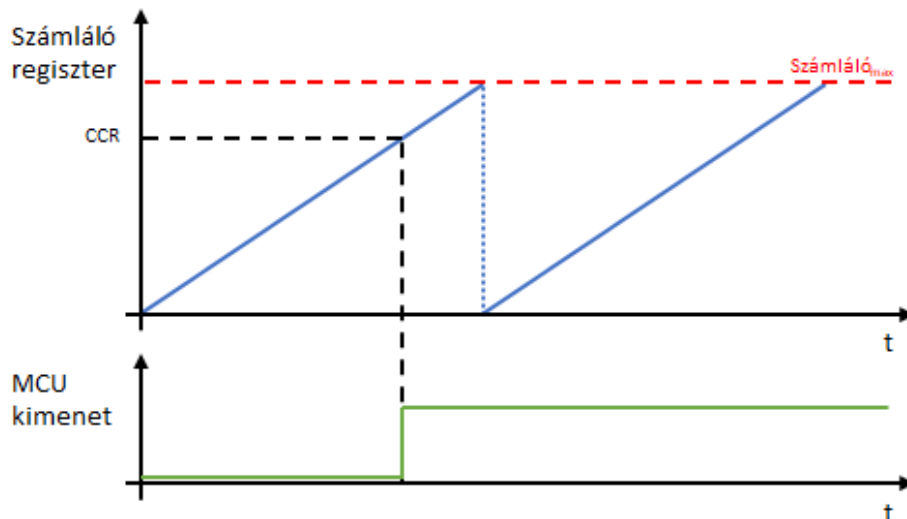


15. ábra: A megszakítás engedélyezése

Ahhoz, hogy létrehozzuk a kívánt jelszint változást a mikrovezérlő lábán, el kell indítanunk a számláló és időzítő modult, hogy a beállított időzítéssel elkezdődjön a számlálás, azaz a kívánt frekvenciával változzon a modul számláló regiszterének értéke. Amint a számláló regiszter értéke eléri a Capture/Compare regiszterben beállított értéket, megtörténik a dedikált lábon a kívánt jelszint beállítása.

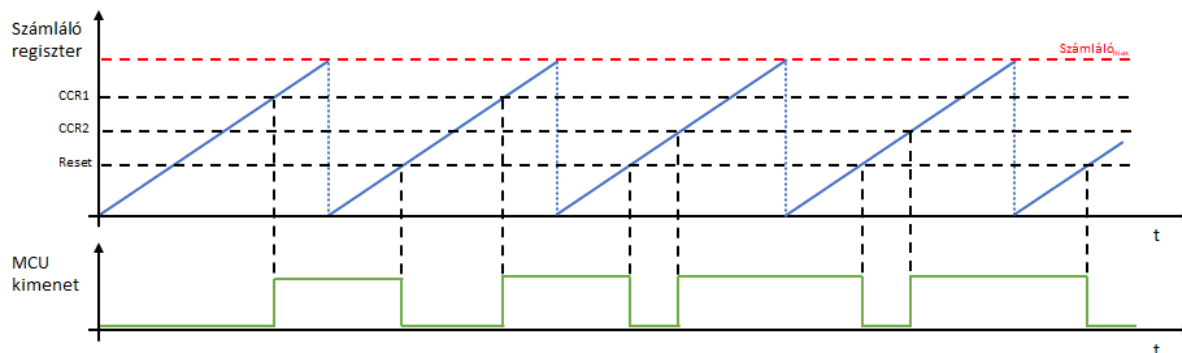
Az alábbi ábra az üzemmód megértését segíti. A mikrovezérlő számláló és időzítő moduljának dedikált lába logikai alacsony jelszintről indul. Amint a számláló regiszter értéke eléri a Capture/Compare

regiszterben (CCR) beállított értéket, jelszint változás történik a kimeneten és logikai magas szint jelenik meg. Amennyiben nem módosítunk a modul beállításain, úgy a kimenet jelszintje változatlan marad. Ezt értelmezhetjük úgy is, hogy a CCR-ben tárolt érték elérése esetén a mikrovezérlő mindig beállítja a megadott (jelen esetben magas) jelszintet, amit csak akkor veszünk észre, ha előtte logikai alacsony szintre állítottuk a kimenetet.



16. ábra: A jelszint megváltozása az időzítés lejártakor

Amennyiben periodikus négyzetjelet szeretnénk generálni az Output Compare üzemmód segítségével, akkor gondoskodunk kell a dedikált láb visszaállításáról is. A megszakításkezelő rutinban eltérő CCR érték beállításával pedig a magas és alacsony jelszintek arányát változtathatjuk. Az alábbi ábrán az első 2 ciklusban a CCR1 limitig számolunk, utána váltunk a CCR2-re. A láb visszaállítása minden esetben a Reset számláló értéknél történik.



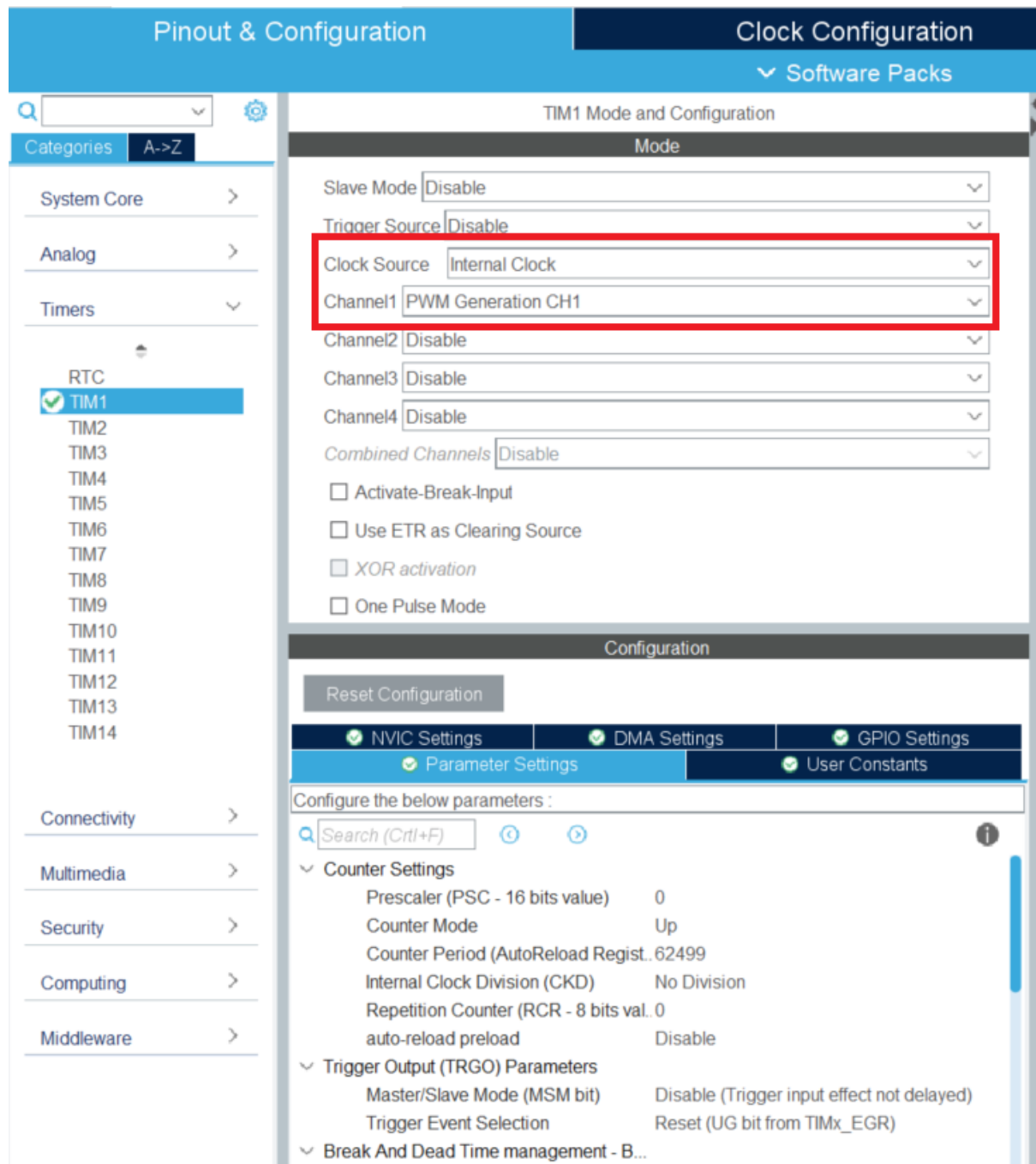
17. ábra: Változó magas és alacsony jelszint arányú négyzetjel generálása

IMPULZUSSZÉLESSÉG-MODULÁCIÓ

Az Output Compare üzemmódnál megismertük, hogy miként lehet pontos időzítés segítségével egymást követő jelszintváltozásokat előidézni egy mikrovezérlő lábon. Ezzel a módszerrel már képesek lehetünk LED-ek fényerejét változtatni vagy megfelelő teljesítményelektronikán keresztül villanymotorokat meghajtani. Azonban nem mehetünk el emellett, hogy ehhez milyen sűrűn hívódnak meg a mikrokontroller megszakításkezelő rutinjai. Ez adott esetben azt is eredményezheti, hogy lemaradunk más, fontos eseményekről a gyakori beavatkozások miatt.

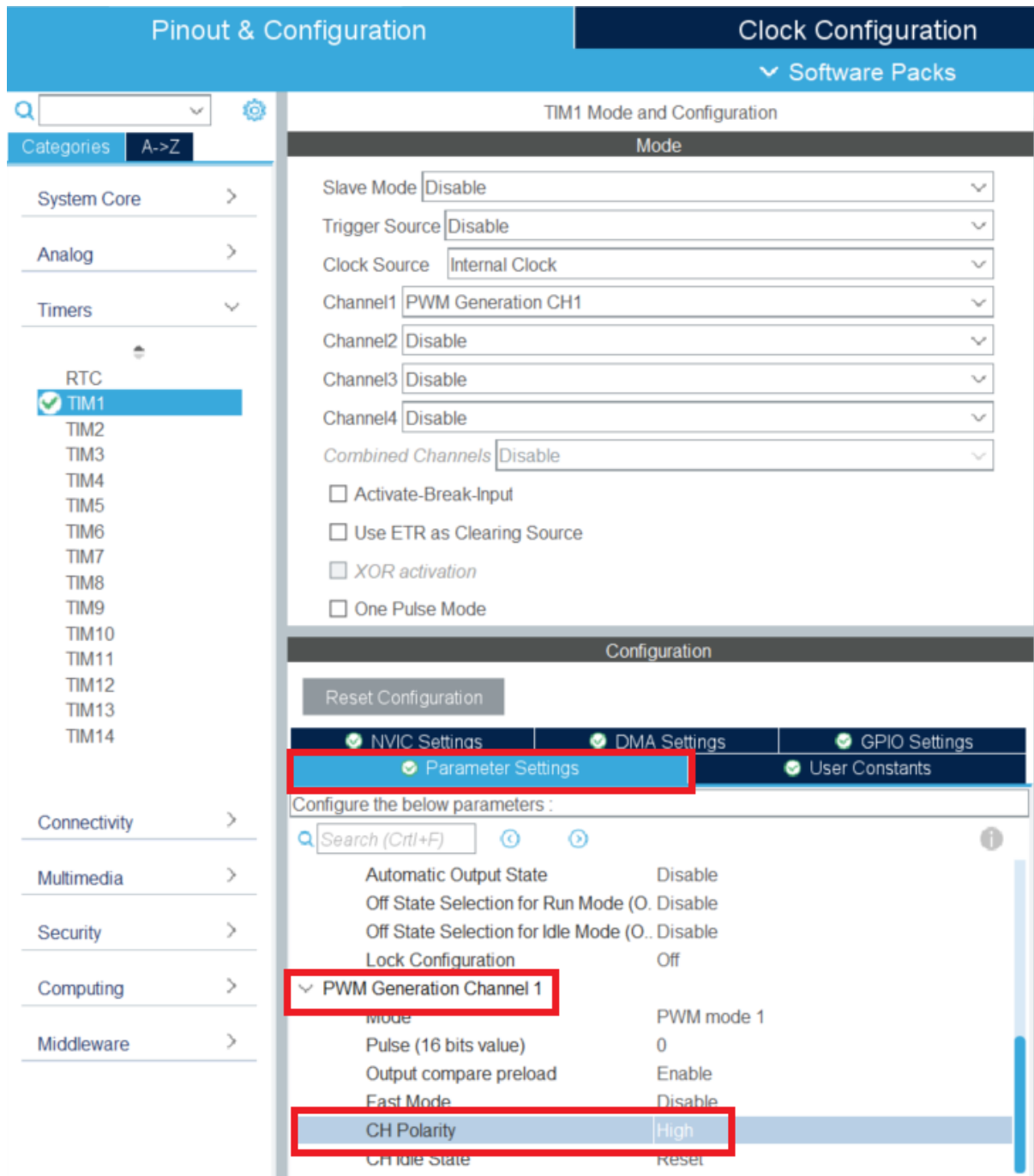
A fenti problémára kínál megoldást a számláló és időzítő modulok impulzus szélesség moduláció (Pulse Width Modulation, vagy röviden: PWM) üzemmódja. A legegyszerűbben úgy képzelhetjük el a működését, hogy a rögzített beállításoknak megfelelően periodikusan megtörténik a mikrovezérlő dedikált lábán a jelszint változás, valamint visszaállítás. Így az automatikus működésnek köszönhetően nem az értékes processzoridőt pazaroljuk a láb mindkét irányba történő mozgatására. A korábban ismertetett üzemmódokhoz hasonlóan vegyük ismét sorra, hogy általában milyen beállítási lépések és lehetőségek vannak PWM mód esetén.

- 1) Engedélyezzük és állítsuk kimenetre a modulunkban a megfelelő Output Compare csatornát, vagyis azt a dedikált lábat, amelyen keresztül beavatkozni kívánunk a rendszerünk működésébe. Példaként a TIM1 esetén állítsuk a Clock Source-ot Internal Clock-ra, a Channel 1-nél pedig válasszuk a PWM Generation CH1 módot.



18. ábra: PWM csatorna kiválasztása

- 2) Be kell állítanunk a dedikált láb polaritását, azaz, hogy az alacsony vagy a magas jelszintet tekintjük aktívnek. Az Output Compare módhoz hasonlóan ebben az esetben is gyakran lehetőségünk van egyszerre több mikrovezérlő láb vezérlésére, melyek aktív polaritását külön-külön beállíthatjuk. Folytatva a példánkat, válasszuk ki a CH Polarity mezőnél a High opciót.



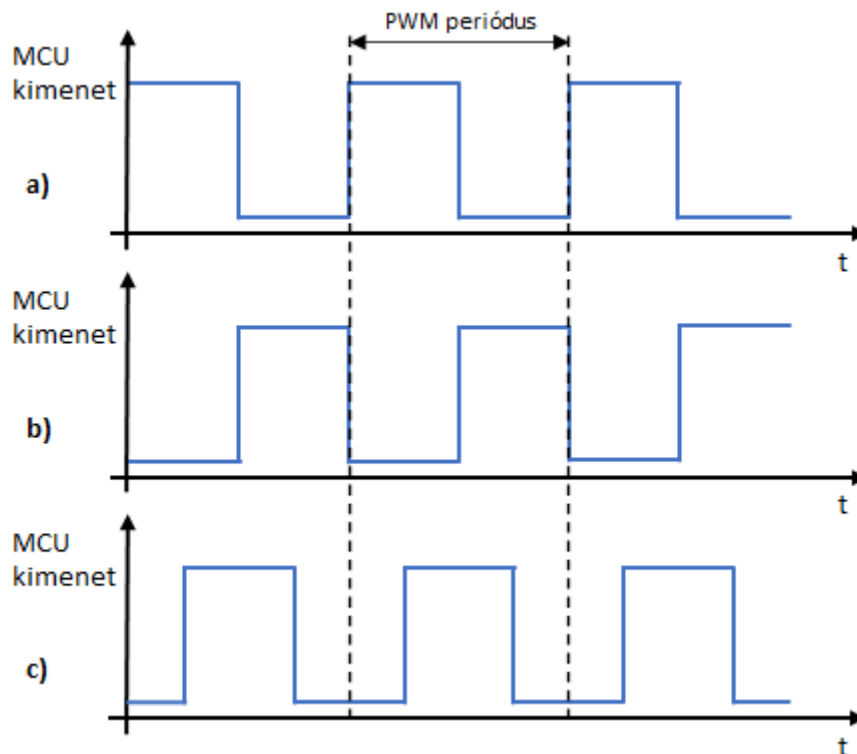
The screenshot shows the STM32CubeMX software interface. On the left, the 'Timers' category is expanded, and 'TIM1' is selected. The main window displays the 'TIM1 Mode and Configuration' settings. In the 'Mode' section, 'Slave Mode' is set to 'Disable', 'Trigger Source' is 'Disable', 'Clock Source' is 'Internal Clock', and 'Channel1' is 'PWM Generation CH1'. The 'Configuration' section shows 'Parameter Settings' highlighted. Under 'PWM Generation Channel 1', 'CH Polarity' is set to 'High'.

19. ábra: A dedikált láb polaritásának beállítása

- 3) Ki kell választanunk a PWM módot, melyet három lehetőség közül tehetünk meg. Amennyiben csak felfelé, vagy csak lefelé számláló modult használunk, akkor lehet a generált PWM jelünk balra vagy jobbra igazított attól függően, hogy magas vagy alacsony jelszinttel kezdődik a PWM periódus. A harmadik módra csak fel/le számláló modul esetén nyílik lehetőségünk. Ilyenkor ugyanis előállíthatunk szimmetrikus jelformát, amelyet villanymotorok vezérlésénél használnak előszeretettel.

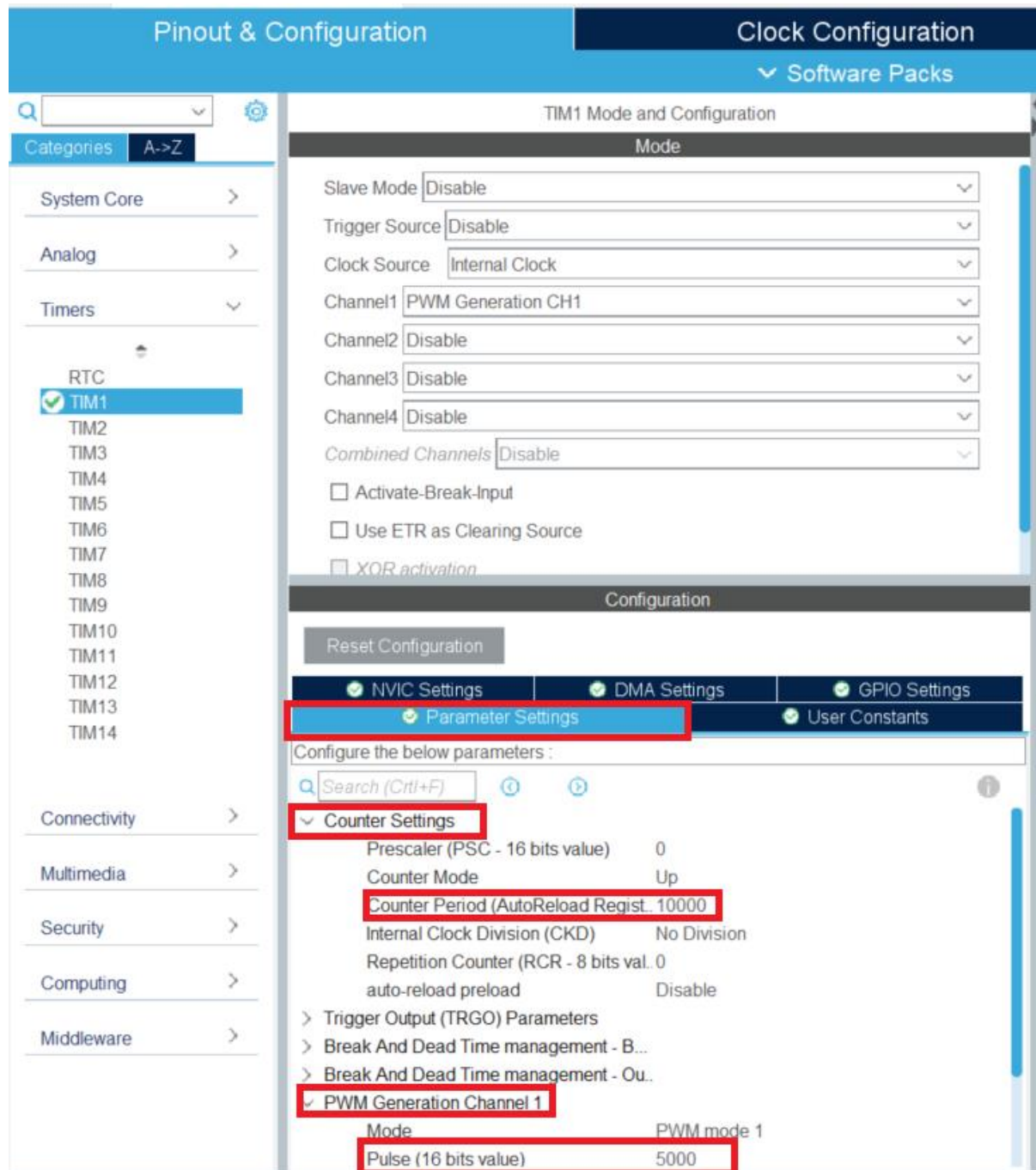
A példánk esetében az előző lépésben egyúttal a PWM módot is meghatároztuk. A modul az aktív polaritás beállításával kezd, ami a logikai magas jelszint, így balra igazított jelformát kapunk.

- 4) Az egyes módokat az alábbi ábrák szemléltetik: az **a)** ábrán balra igazított, a **b)** -n jobbra igazított, míg a **c)** -n szimmetrikus PWM jelformát láthatunk.



20. ábra: Balra igazított, jobbra igazított és szimmetrikus PWM jel

- 5) El kell végeznünk azon két regiszter beállítását, melyek a jelalakot határozzák meg. Egyik ezek közül a számláló üzemmódból ismerős számlálás határát nyilvántartó regiszter, amely a generált PWM jel periódusidejét befolyásolja. A másik pedig az Output Compare csatornához tartozó Capture/Compare regiszter, amellyel azt állíthatjuk be, hogy a PWM perióduson belül hol történjen meg a jelszint váltása, azaz a jel kitöltési tényezőjét határozza meg.



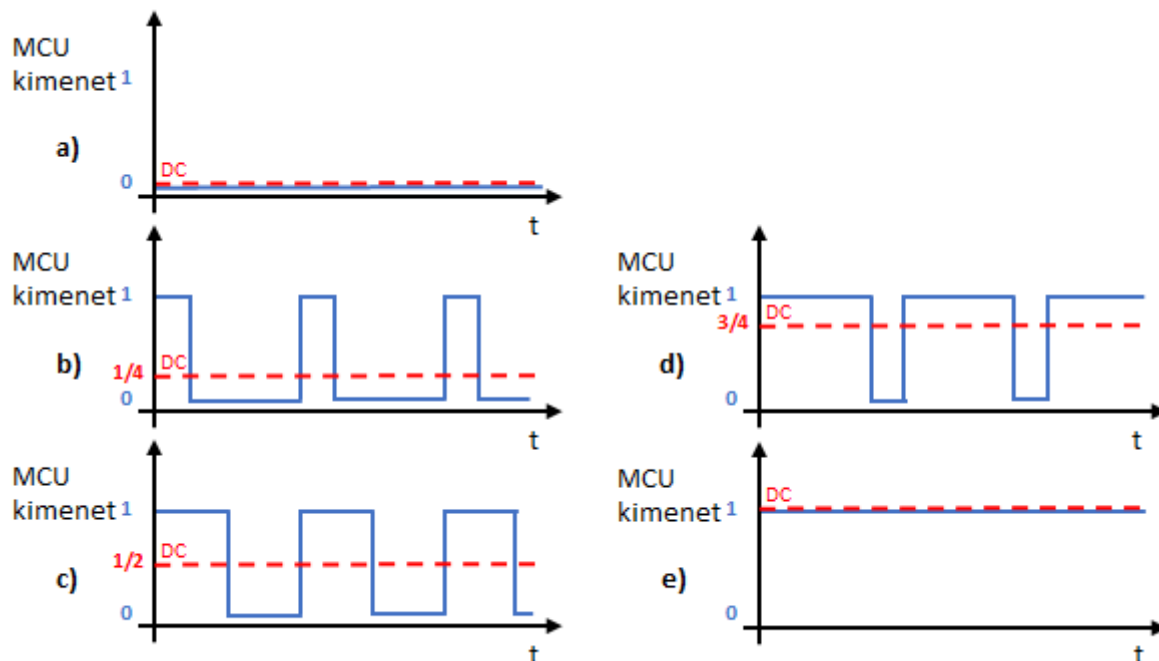
21. ábra: A kitöltési tényező beállítása

A példa kedvéért 50%-os kitöltési tényezőjű PWM jelet generálunk, a számlálás határát a Counter Period mezőnél 10 000-re választottam. Ennek az értéknek pontosan a felét kell a Pulse paraméterhez írni (az 50%-os kitöltési tényező miatt), ugyanis ez határozza meg a jelszint váltásának pillanatát.

- 6) Végül pedig engedélyezhetünk megszakítást, hogy értesüljünk a jelszint változásokról, azonban PWM mód esetén ezt jellemzően nem használjuk, hiszen a periodikus jel generálása folytatódik a beállításoknak megfelelően.

A beállított jelforma periodikus generálásához el kell indítanunk a számláló és időzítő modult. A PWM jellel vezérelt áramkörre gyakorlatilag egy átlagérték kerül, melynek értéke feszültségben kifejezve a PWM perióduson belüli alacsony és magas jelszintek időtartamának arányától, azaz a kitöltési tényezőtől függ.

Az előzőekben leírtakat szemléltetik az alábbi ábrák, amelyeken a logikai alacsony jelszint 0 V (0), a magas jelszint a 3,3 V (1). Az **a)** ábrán 0%-os kitöltési tényezőjű jelet láthatunk, ilyenkor a kimeneten is 0 V jelenik meg. A **b)** ábrán a kitöltési tényező 25%, a **c)** ábrán a kitöltési tényező 50%, míg a **d)** esetén 75%. Az **e)** ábrán 100%-os a kitöltési tényező, ilyenkor 3,3 V feszültség mérhető a mikrokontroller lábán.



22. ábra: Példák különböző kitöltési tényezőjű PWM jelekre

A gyakorlatban rengeteg példát találhatunk a PWM jel alkalmazására vezérlés céljából. Használhatjuk fényerő változtatására, például monitorok, televíziók, telefonok képernyője vagy LED alapú dekorációk esetén.

Előszeretettel alkalmaznak PWM jelet villanymotorok vezérlésére, például a számítógépekben található ventilátorok esetén, de találkozhatunk vele az elektromos járművekben alkalmazott villanymotorok szabályozókörében is. Erről bővebben egy későbbi tananyagrészen olvashatunk majd.

Az impulzusszélesség-modulációról további hasznos ismeretanyag található a Kristálytisza Elektronika 15. tananyag részében.

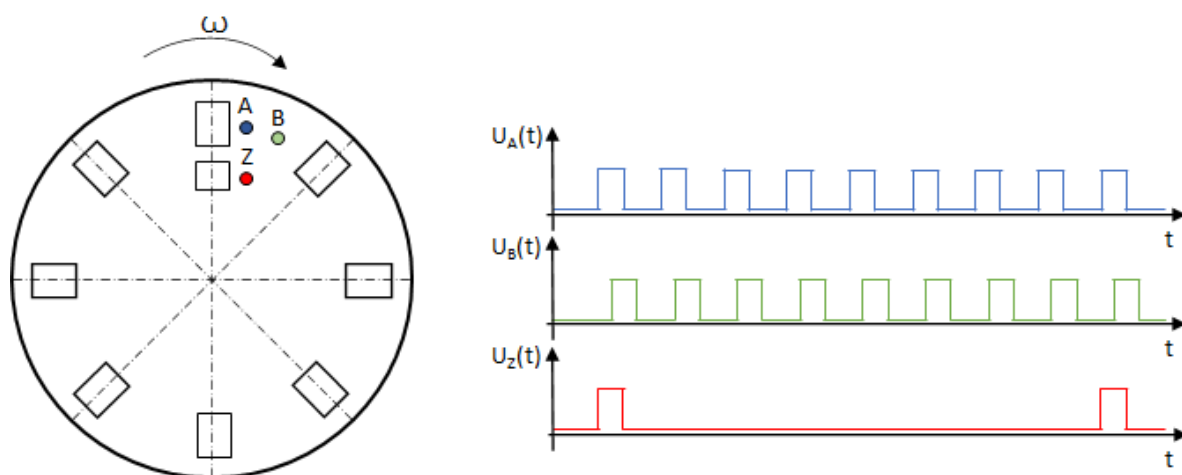
ENKÓDER INTERFÉSZ

Az inkrementális enkóder, vagy más néven forgójeladó, az adott mértékű szögelfordulást meghatározott számú impulzussal jelez, amelyek számolásával meghatározhatjuk a forgó tengelyünk relatív pozícióját. Ilyen szenzorokkal találkozhatunk például számítógéppel vezérelt gyártósorok és munkaállomások esetében. Ezekben a berendezésekben (illetve általában véve a villanymotorokban)

a forgási jellemzőket előszeretettel mérik ezen a technológián alapuló eszközökkel, a mikrovezérlők pedig gyakran támogatják az ilyen szenzoroktól származó jelek feldolgozását, így érdemes megismerkedni a számláló és időzítő modulok ezen funkciójával is.

Az Input Capture üzemmód bemutatásánál megismerhettük, hogy a Timer/Counter modulok miként alkalmazhatók a mikrovezérlőn kívül bekövetkező események detektálására. Egy enkóder jeleinek feldolgozásához pedig egy ehhez nagyon hasonló feladatot kell végrehajtani.

A forgójeladók, a teljesség igénye nélkül, lehetnek mágneses, mechanikai vagy optikai elven működők. Példaként ismerkedjünk meg egy optikai elven működő inkrementális jeladóval, melyhez az alábbi ábra nyújt segítséget.



23. ábra: Optikai elven működő inkrementális jeladó elvi vázlata

Képzeld el, hogy az ábrán látható tárcsa állandó szögsebességgel forog az óramutató járásával megegyező irányban. **A**, **B** és **Z** pedig fototranzisztorok, melyek a tárcsa túloldalán található fényforrás által kibocsátott fényt érzékelik. A tárcsán egyenletes elosztásban 8 darab ablak található, melyeken keresztül **A** és **B** érzékelné a párjától érkező fényt. **Z** pedig csak egy ablakon képes fényt detektálni, ez a tárcsa nulla pontja.

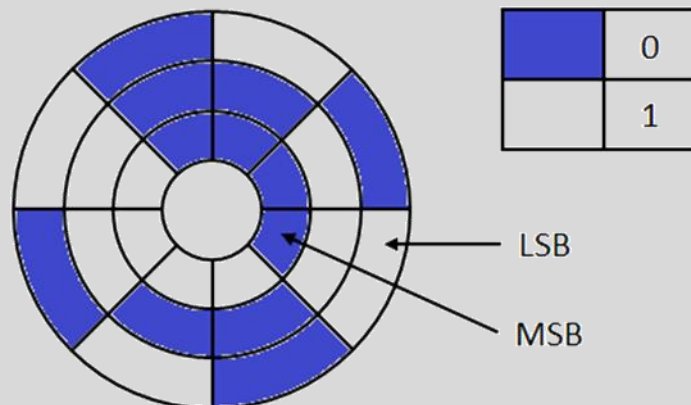
A tárcsa forgása alapján a fenti diagramok szerint alakulnak **A**, **B** és **Z** kimenetei: amikor érzékelik a fényt, logikai magas jelszintű a kimenet, ellenkező esetben pedig alacsony. Megfigyelhetjük az **A** és **B** kimenetek közötti fáziseltolódást, amely a forgásirány meghatározásában segít. Képzeld csak el: amennyiben az ellenkező irányba forogna a tárcsa, akkor a **B** kimenet egy kicsivel előbb váltana magas jelszintre, mint az **A**.

Az ábrán látható tárcsát felszerelhetjük egy forgó tengely egyik végére és máris figyelemmel kísérhetjük a tengelyünk forgását.

Kitekintés:

Találkozhatunk abszolút jeladókkal is, melyek előnye az inkrementális változattal szemben az, hogy a pozíció meghatározásához nincs szükség az impulzusok számlálására és az eszköz újraindítása után rögtön meghatározható a pontos pozíció. Ehhez a forgó tengelyre egy tárcsát erősítenek, amely koncentrikus gyűrűkre van felosztva. Optikai elven működő jeladó esetén a gyűrűk váltakozva tartalmaznak fényt átengedő vagy kitakaró szegmenseket. A tárcsa középpontjától a széléig leolvasva előáll a pozíció kódja, amely lehet például bináris vagy Gray-

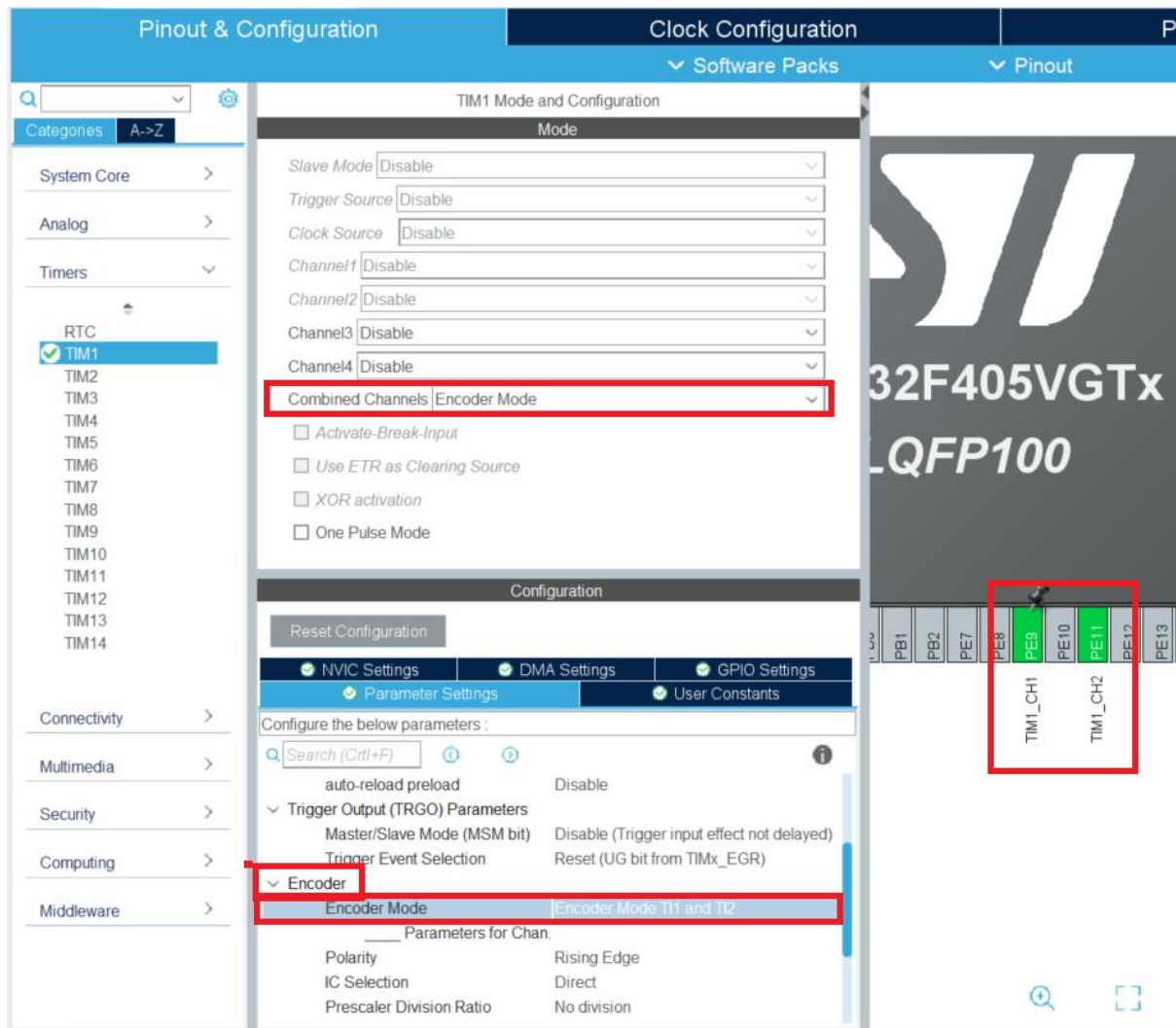
kód. Utóbbi esetében az egymás követő kódszavak csak 1 bitben térnek el (két bites kódszó esetén a sorrend a következő: 00-01-11-10-00). Egy bináris kódolású tárcsát szemléltet az alábbi ábra.



24. ábra: Bináris kódolású abszolút jeladó elvi vázlata

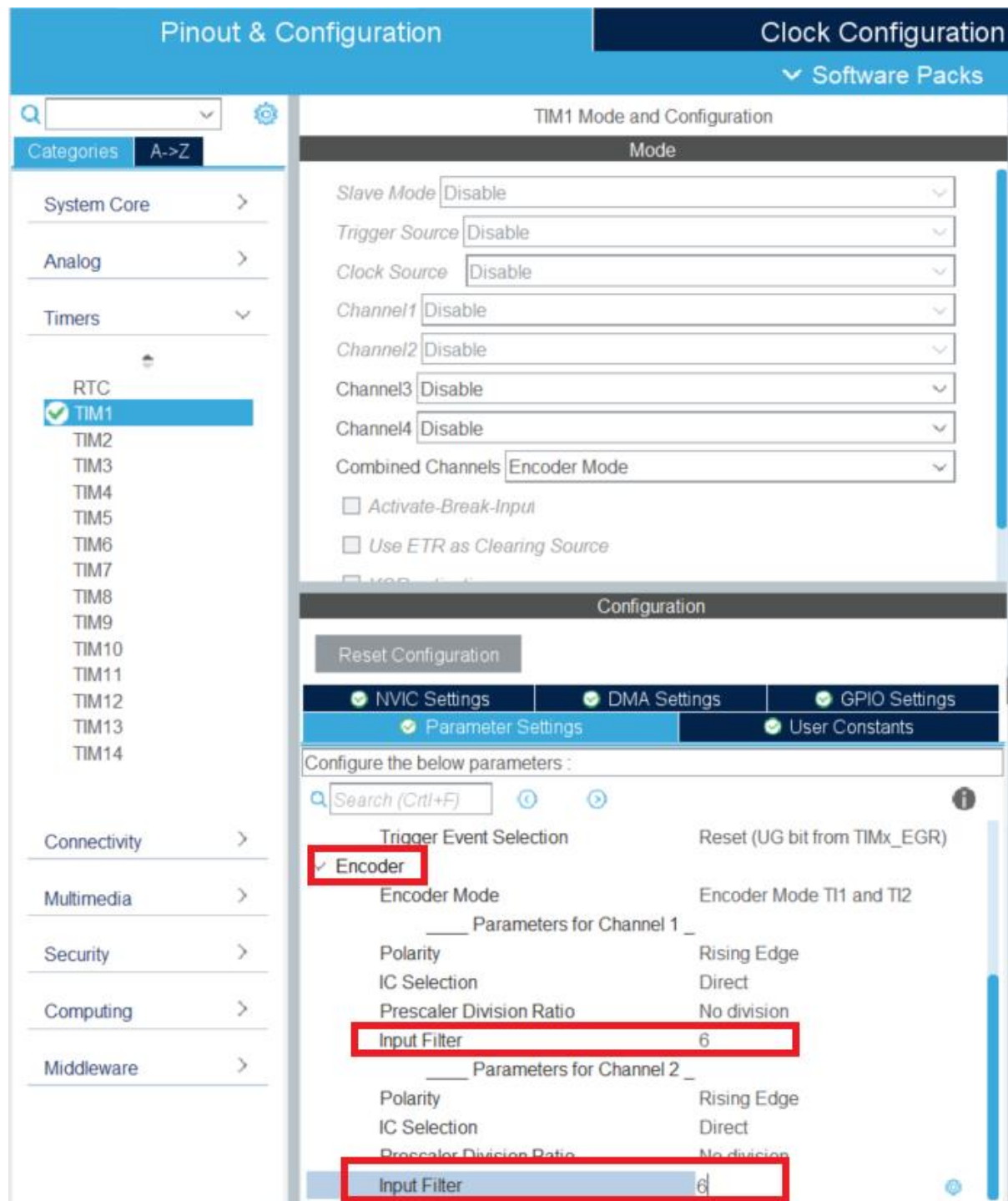
A korábbi üzemmódokhoz hasonlóan vegyük sorra, hogy milyen beállításokkal dolgozhatunk.

- 1) Encoder interface üzemmódban be kell állítanunk, hogy mennyi bemenettel dolgozunk. Általában használhatjuk csak az **A**, csak a **B** vagy **A** és **B** jeleket egyaránt. Természetesen ezeket a jeleket a számláló és időzítő modulunk megfelelő lábaira kell kötni. A kétbemenetes üzemmódot beállíthatjuk például a TIM1 esetén, ha a Combined Channels mezőben az Encoder Mode-ot választjuk. A Parameter Settings-en belül az Encoder Mode-nál pedig az Encoder Mode TI1 and TI2 lehetőséget állítjuk be.



25. ábra: Bemenetek kiválasztása

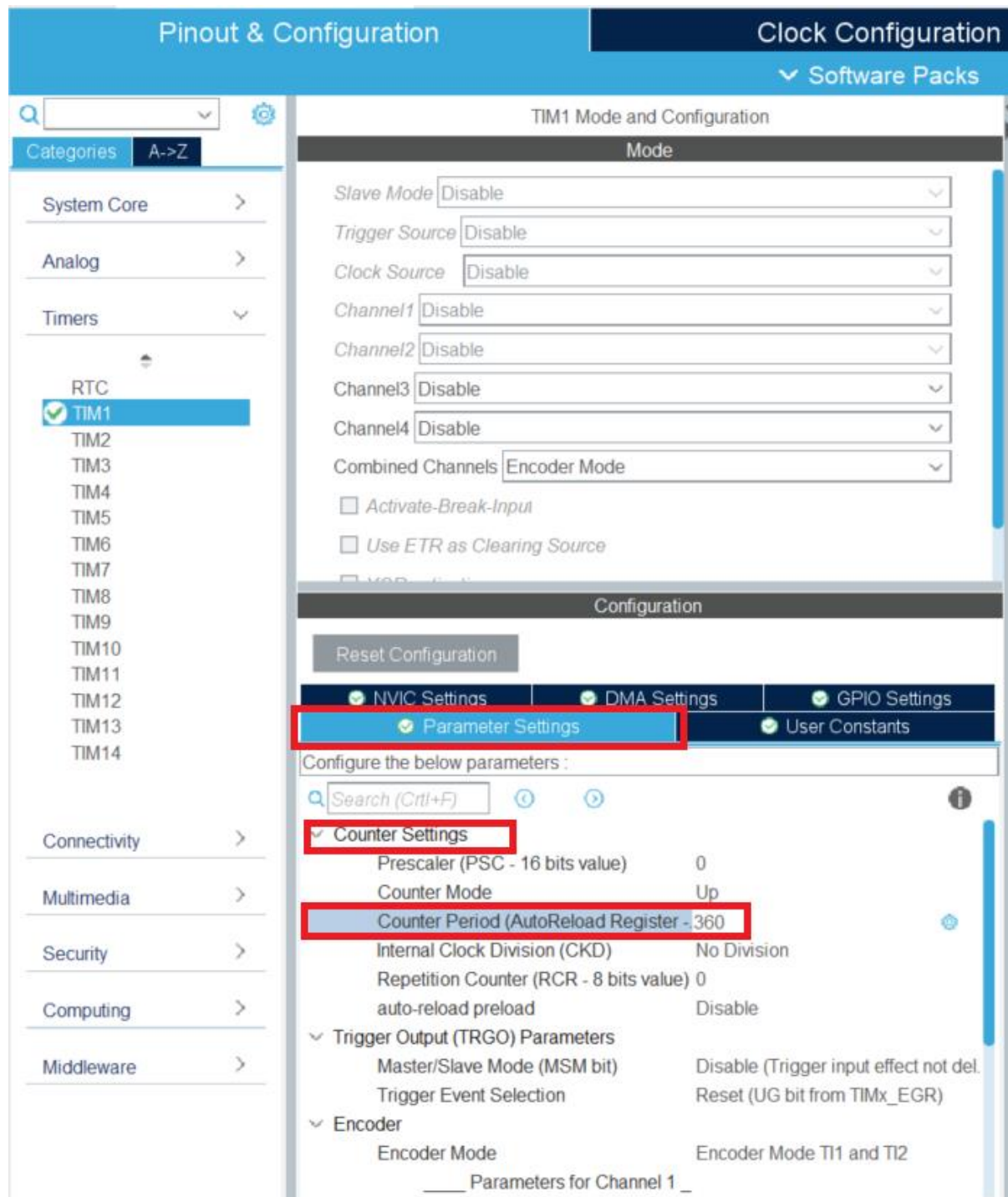
- 2) Amennyiben használni kívánjuk a nulla pozíció jelét (Z), úgy azt beköthetjük egy általános GPIO lábra, melyen External Interrupt-ot állítunk be, ahogy azt a tananyagban használt mikrokontroller gyártója is javasolja.
- 3) Konfigurálni kell a bemenetek aktív polaritását és adott esetben bemeneti szűrést is alkalmazhatunk. A példa kedvéért mindkét bemeneten lefutó élre történő detektációt állítottam be, a bemeneti szűrésnél pedig 6-6 minta figyelembevételét.



26. ábra: Lefutó élre történő detektáció, valamint bemeneti szűrés beállítása

- 4) Ki kell választanunk, hogy melyik bemeneten szeretnénk számoltatni az enkóder impulzusait. Adott esetben megtehetjük ezt több bemenet felfutó/lefutó vagy mindkét élére is. A jelenlegi beállítások mellett mindkét bemenet lefutó élére számolunk. Amennyiben csak az egyik alapján szeretnénk számoltatni, válasszuk az Encoder Mode TI1-et vagy az Encoder Mode TI2-őt.
- 5) Be kell állítanunk a számlálás határát az erre a célra fenntartott regiszterben. A számláló regiszter ugyanis 0 és a határ között számol fel vagy le, a forgás irányának megfelelően. A

példa kedvéért használjunk egy olyan enkódert, amely 1° szögelfordulásonként ad impulzust, így ezt az értéket 360-ra állítottam.



27. ábra: A számlálás határának beállítása

6) Végül pedig el kell indítanunk a számláló és időzítő modult.

A számláló regiszter mindig az aktuális pozíciónak megfelelő értéket fogja tartalmazni. A forgás irányáról általában találunk információt a számláló és időzítő modulok regisztereinek egyikében,

melyet a mikrokontroller magától frissít a bemenetek alapján. Amennyiben használjuk a referenciapontot jelző szenzor jelet, akkor a számláló regiszter értékébe történő beavatkozást minden körbefordulás során ugyanazon a ponton tehetjük meg.

Fontos megjegyezni, hogy az Encoder interface csak az aktuális pozícióról szolgáltat információt, azaz jelen esetben a tengelyünk szöghelyzetéről. Más, dinamikus jellemzők (például szögsebesség, szöggyorsulás) megállapításához egyéb megoldásokhoz kell folyamodnunk. Például a szögsebességet meghatározhatjuk úgy, hogy az enkóder referencia pontjánál meghívódó External Interrupt rutinban elindítunk egy másik számláló modult és mérjük a következő referenciapont eléréséig eltelt időt.

A WATCHDOG TIMER

A bevezető részben említésre került a Watchdog, mint önálló periféria, amely megvédi az esetleges szoftverhibákból adódó meghibásodásoktól, megakadásoktól, ezáltal elősegítve rendszerünk való idejű működését.

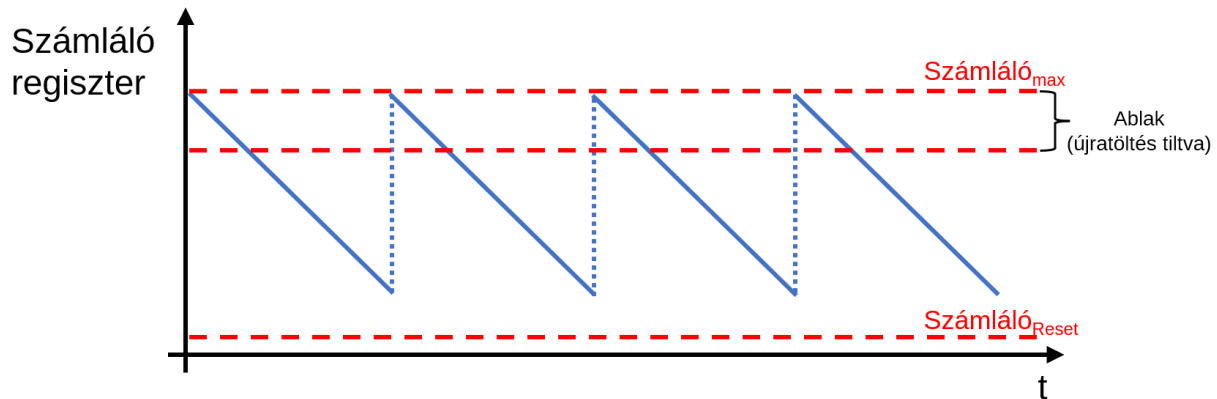
Általában véve a Watchdog egy folyamatosan lefelé számláló időzítő, amely újraindítást generál, ha elérte a 0 értéket, azaz lejárt az időzítése. Ahhoz, hogy ezt elkerüljük, megfelelő időközönként vissza kell állítanunk a számlálót egy magasabb értékre. Így, ha megakadna a szoftverünk futása, feltételezhetjük, hogy a Watchdog időzítő újrakódolása sem történik meg. Azonban a lejárt időzítés miatt újraindul a programunk, ezzel kibillentve rendszerünket a meghibásodás okozta megakadásból.

Jogosan tehetjük fel a kérdést, hogy mi történik abban az esetben, ha szerencsétlen módon végtelen ciklusba kerül a szoftverünk, ráadásul pont úgy, hogy rendszeresen megtörténik a Watchdog újra felhívása is. Szerencsénkre a mikrovezérlő gyártók erre is gondoltak, ezért az általános Watchdog modul általában vagy kiegészítik egy ablakos funkcióval (ennek működéséről a későbbiekben esik szó), vagy egy teljesen független periféria formájában bocsátják rendelkezésünkre.

A Watchdog perifériák egyszerű működési mechanizmusuk miatt általában csak néhány konfigurálási lehetőséggel rendelkeznek, vegyük is sorra ezeket!

- 1) A mikrokontroller gyártók részéről bevett gyakorlat, hogy biztonsági okokból a Watchdog modul bizonyos regisztereit csak a helyes jelszó megadása után lehet módosítani, különben nem lépnek érvénybe a beállításaink. Ezt a jelszót egy dedikált regiszterbe kell beírni, amelyről bővebben a kézikönyv 708. oldalán lehet olvasni.
- 2) A hagyományos számláló és időzítő modulokhoz hasonlóan a Watchdog esetében is lehetőségünk van különféle előosztásokat beállítani, ezáltal az időzítés hosszát módosítani.
- 3) A Watchdog modulok esetén is rendelkezésünkre áll egy regiszter, amelyben számlálási limitet állíthatunk be. Természetesen ebben ugyanúgy korlátoz minket a számláló felbontása. A Watchdog felhívásához elterjedt az a megoldás, hogy a már említett jelszó regiszterbe kell beírni az újratöltés jelszavát. Ennek hatására a számlálási limitet tartalmazó regiszter értéke átmásolásra kerül a számláló regiszterbe, ezzel újra feltöltve az időközben lecsökkent számláló regiszter értékét.
- 4) Bizonyos esetekben a Watchdog kiegészíthető egy úgynevezett ablakkal, amely azt a célt szolgálja, hogy az időzítés lejártá mellett a túl gyakori újratöltés hatására is újraindítást generál. Ablakos Watchdog modul esetén be kell még állítanunk az ablak határát, amivel

meghatározhatjuk, hogy legalább mennyi időt kell várnunk két számláló újrahúzás között. Az alábbi ábra szemlélteti az ablakos Watchdog működését. Amennyiben a tiltott tartományban kíséreljük meg újrahúzni a számlálót, akkor azonnal újraindulást generál a periféria. A hagyományos Watchdog működése ehhez képest annyiban tér el, hogy bármikor elvégezhetjük a számláló újratöltését és újraindulás csak akkor történik, ha ezt elmulasztjuk és eléri a 0-át a számláló.



28. ábra: Az ablakos Watchdog működése

SYSTICK TIMER

Összetett feladatokat ellátó beágyazott rendszereinkben gyakran alkalmazunk valamilyen beágyazott operációs rendszert a feladatok ütemezéséhez. Az operációs rendszer valós idejű működése azonban nem biztosítható pontos időzítés nélkül. Erről a 21.-22. tananyag részben olvashattok bővebben.

A legtöbb esetben a mikrovezérlőkön futó operációs rendszer egy dedikált számláló és időzítő modult, a SysTick Timer-t (rendszeridőzítő) használja az ütemezési feladatok ellátásához. Az említett időzítő pedig megtalálható szinte az összes Cortex-M alapú mikrovezérlőben, amelyeket kifejezetten beágyazott rendszerekbe történő alkalmazásra fejlesztenek.

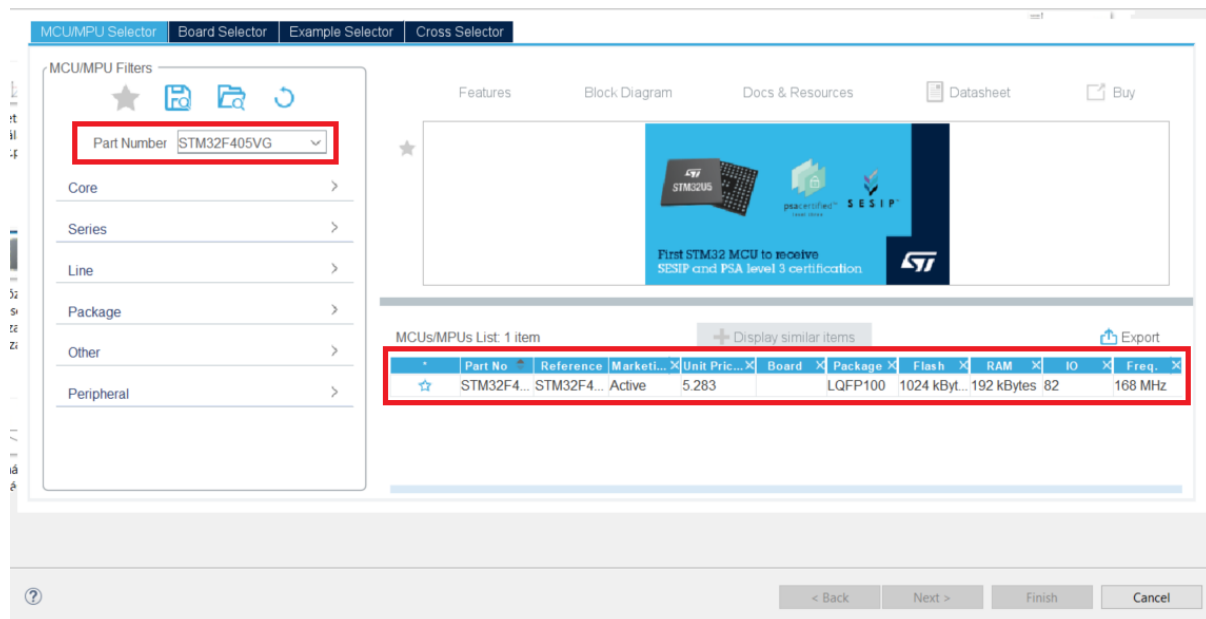
A SysTick-ről annyit érdemes tudni, hogy ez egy 24 bit felbontású, lefelé számláló modul, melyet a beágyazott operációs rendszerek az inicializálás fázisában állítanak be. Az időzítését az operációs rendszer konfigurációs paraméterei között módosíthatjuk, amelynek lejártakor lefut az operációs rendszer ütemezője és eldönti, hogy melyik feladatot (task-ot) kell futtatnia.

PÉLDÁK A GYAKORLATBAN TÖRTÉNŐ ALKALMAZÁSRA

LED VILLOGTATÁSA TIMER INTERRUPT SEGÍTSÉGÉVEL

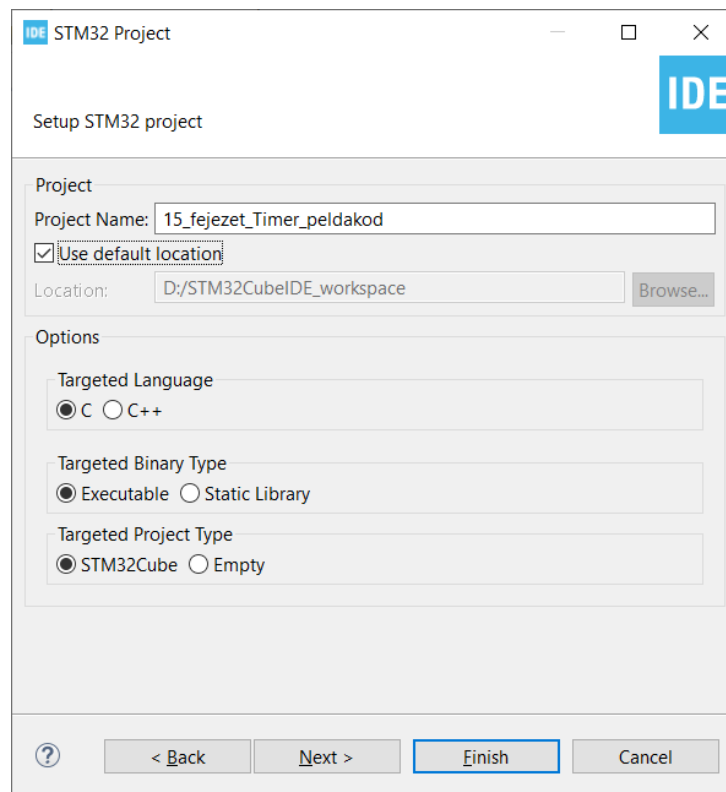
Ebben a példában a fejlesztői panelen található LED1-et fogjuk villogtatni egy számláló és időzítő modul segítségével, 1 másodpercenként. A továbbiakban lépésről lépésre ismertetem a példakód működését és a szükséges beállításokat.

- 1) Első lépésként hozzunk létre egy új projektet az STM32CubeIDE fejlesztői környezetben a panelünkön található mikrovezérlőhöz. Ezt megtehetjük a Fájl → New → STM32 Project-re kattintva. A Part Number legördülő menüből válasszuk ki az STM32F405VG mikrokontrollert és kattintsunk rá az MCUs/MPUs List-ben is, majd a pedig a Next gombra.



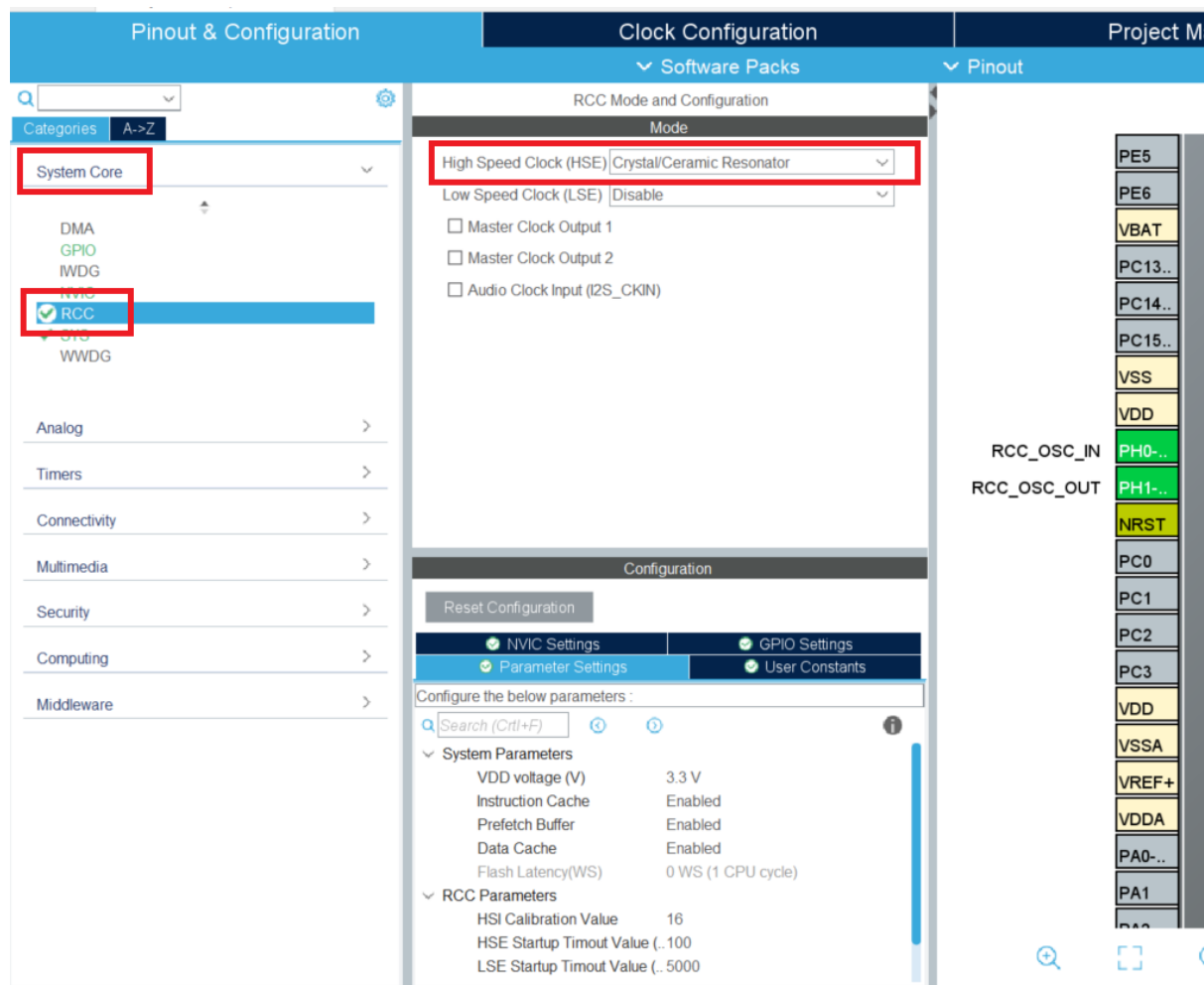
29. ábra: Új projekt létrehozása

- 2) Nevezzük el a projektünket, majd kattintsunk a Finish-re. Ezzel el is készül a konfigurációs fájl, a következő néhány lépés során ebben fogunk dolgozni.



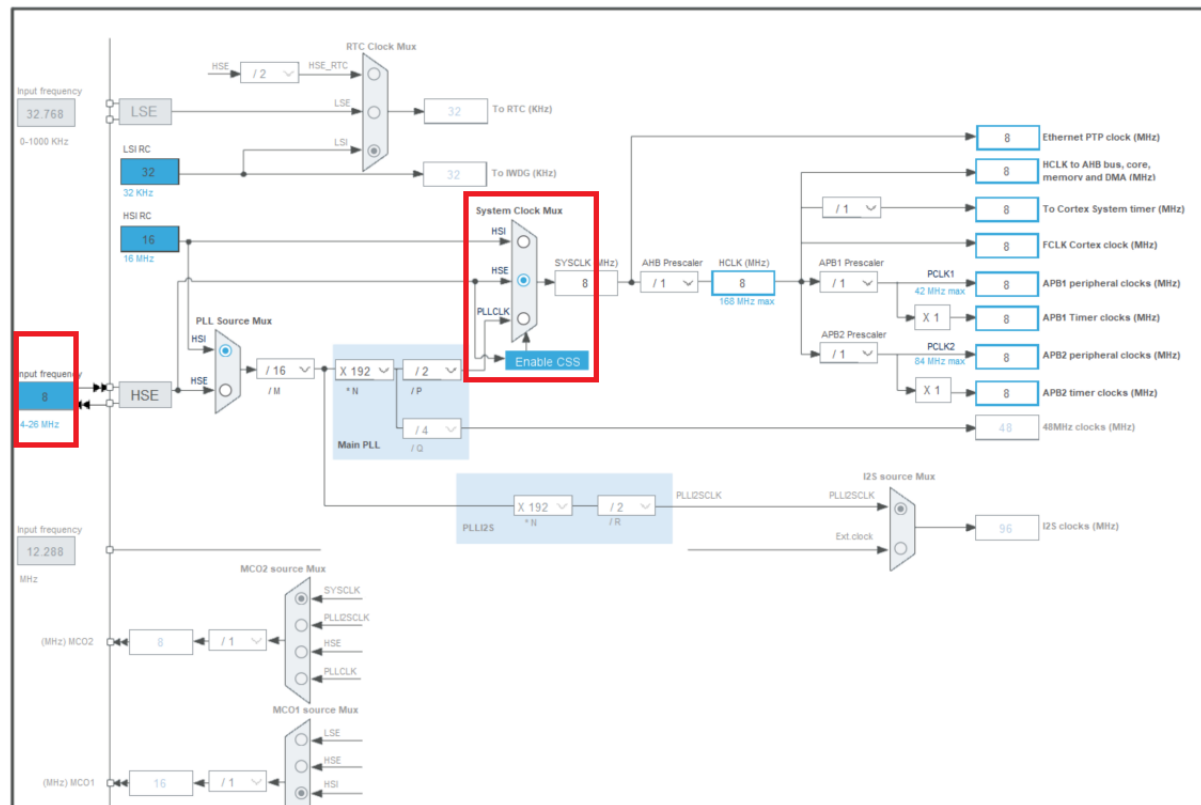
30. ábra: A projekt nevének megadása

- 3) A System Core menüből válasszuk ki az RCC-t és a High Speed Clock (HSE) beállítását változtassuk meg Crystal/Ceramic Resonator-ra. Erre azért van szükség, mert jellemzően a külső órajelforrások használatával jóval pontosabb órajelet lehet előállítani a mikrovezérlőbe integrált oszcillátorokhoz viszonyítva, ami komoly hatással van az időzítéseinkre is.



31. ábra: Külső órajelforrás beállítása

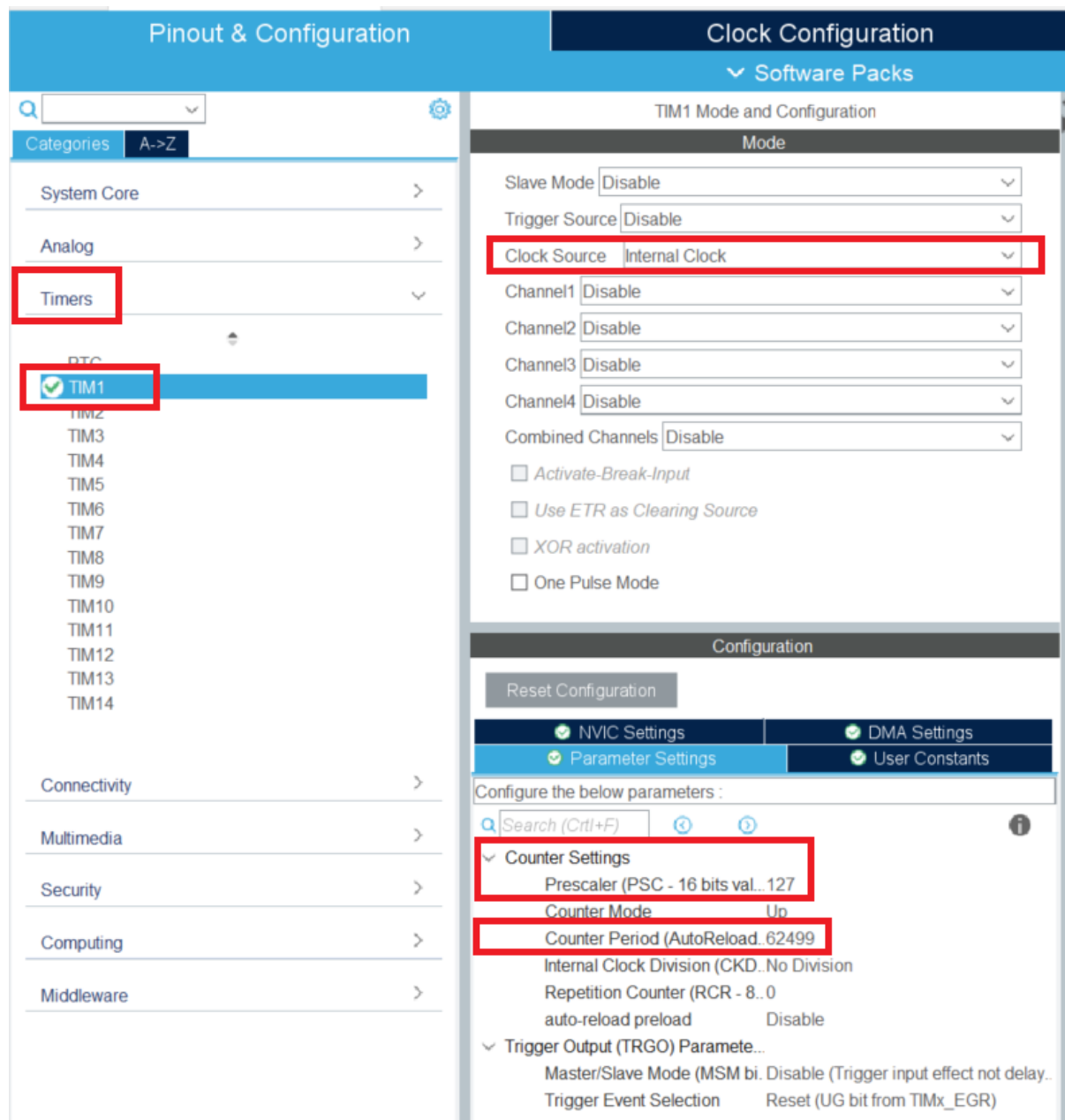
- 4) A további órajel beállításokat a Clock Configuration fülön végezzük el. Át kell írunk az Input frequency-t 8 MHz-re, hiszen a fejlesztőpanelen ekkora frekvenciájú a külső órajelforrás. A System Clock Mux-nál válasszuk ki a HSE lehetőséget. Ezzel írjuk elő, hogy a számláló és időzítő modulok által használt APB1 és APB2 órajelek is a külső forrást vegyék alapul.



32. ábra: Az órajel további beállításai

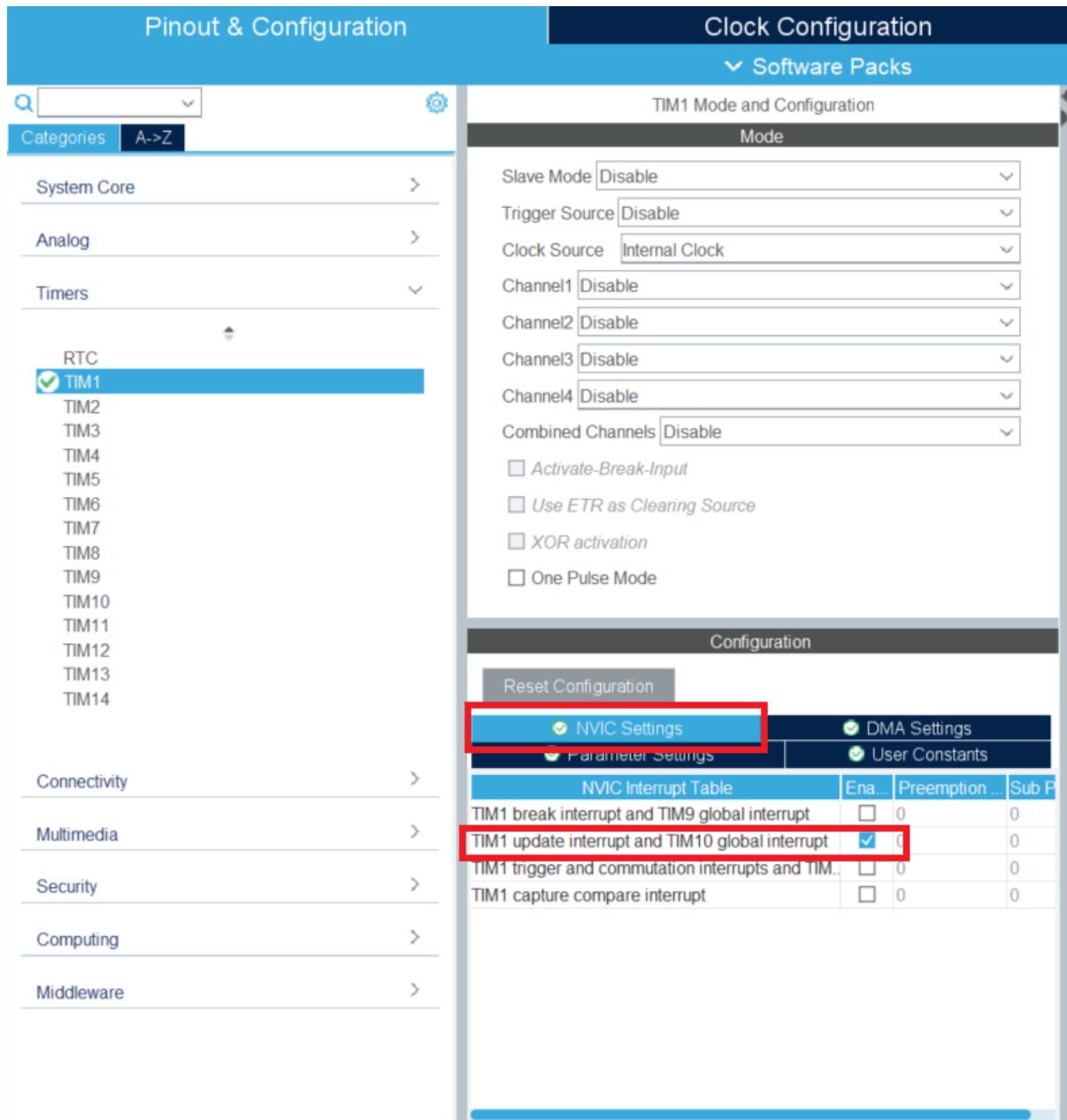
- 5) Térjünk vissza a Pins & Configuration fülre és a Timers menüből kattintsunk a TIM1-re. A Clock Source-ot állítsuk Internal Clock-ra, így a TIM1 modul a mikrovezérlő által előállított órajelből fog dolgozni. Az 1 másodperces időzítést a következő beállításokkal érhetjük el. A Parameter Settings-en belül írjuk át az előosztást (Prescaler) 127-re. A tényleges leosztás ebben az esetben 128 lesz, ugyanis a Prescaler regisztert 0-ra állítva jutunk a leosztás nélküli, eredeti órajelhez. Erről bővebben a kézikönyv 580. oldalán olvashatunk. A számlálási határt (Counter Period) 62 499-re konfiguráljuk. Ebben az esetben azt kell szem előtt tartanunk, hogy az interrupt esemény akkor generálódik, amikor a modul elszámol a beállított értékig, majd átfordul 0-ra, azaz további 1 órajel ciklus telik el. Az időzítés hosszát ellenőrizhetjük a következő összefüggés alapján:

$$T [s] = \frac{\text{Előosztás} \times \text{Limit}}{f_{\text{periféria}} [Hz]} = \frac{(127 + 1) \times (62\,499 + 1)}{8\,000\,000 [Hz]} = 1 [s].$$



33. ábra: Az időzítés hosszának beállítása

- 6) Az NVIC Settings-en belül pipáljuk ki a TIM1 update and TIM10 global interrupt lehetőséget, hogy értesítést kapjunk a beállított időzítés lejárta.



Pinout & Configuration

Categories A->Z

- System Core
- Analog
- Timers
 - RTC
 - TIM1**
 - TIM2
 - TIM3
 - TIM4
 - TIM5
 - TIM6
 - TIM7
 - TIM8
 - TIM9
 - TIM10
 - TIM11
 - TIM12
 - TIM13
 - TIM14
- Connectivity
- Multimedia
- Security
- Computing
- Middleware

Clock Configuration

Software Packs

TIM1 Mode and Configuration

Mode

Slave Mode: Disable

Trigger Source: Disable

Clock Source: Internal Clock

Channel1: Disable

Channel2: Disable

Channel3: Disable

Channel4: Disable

Combined Channels: Disable

☐ Activate-Break-Input

☐ Use ETR as Clearing Source

☐ XOR activation

☐ One Pulse Mode

Configuration

Reset Configuration

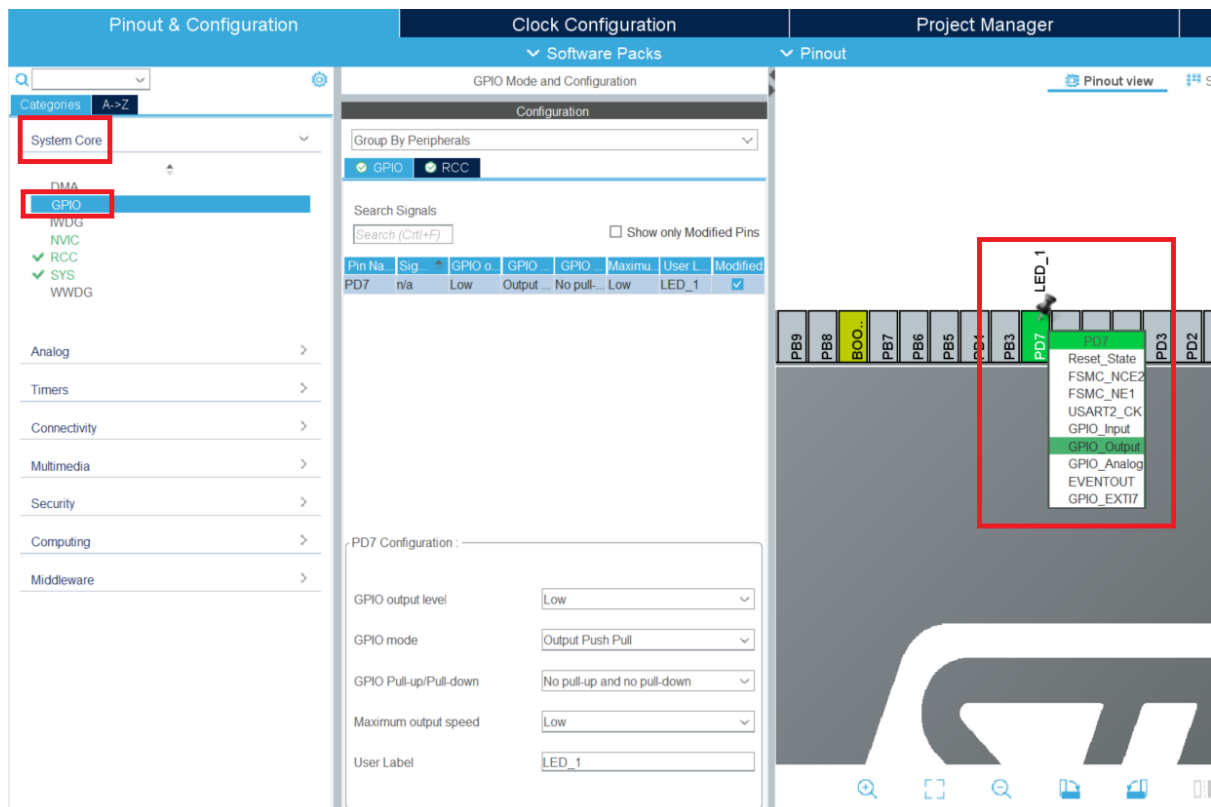
NVIC Settings

NVIC Interrupt Table

Interrupt	Ena...	Preemption ...	Sub P
TIM1 break interrupt and TIM9 global interrupt	☐	0	0
TIM1 update interrupt and TIM10 global interrupt	☒	0	0
TIM1 trigger and commutation interrupts and TIM...	☐	0	0
TIM1 capture compare interrupt	☐	0	0

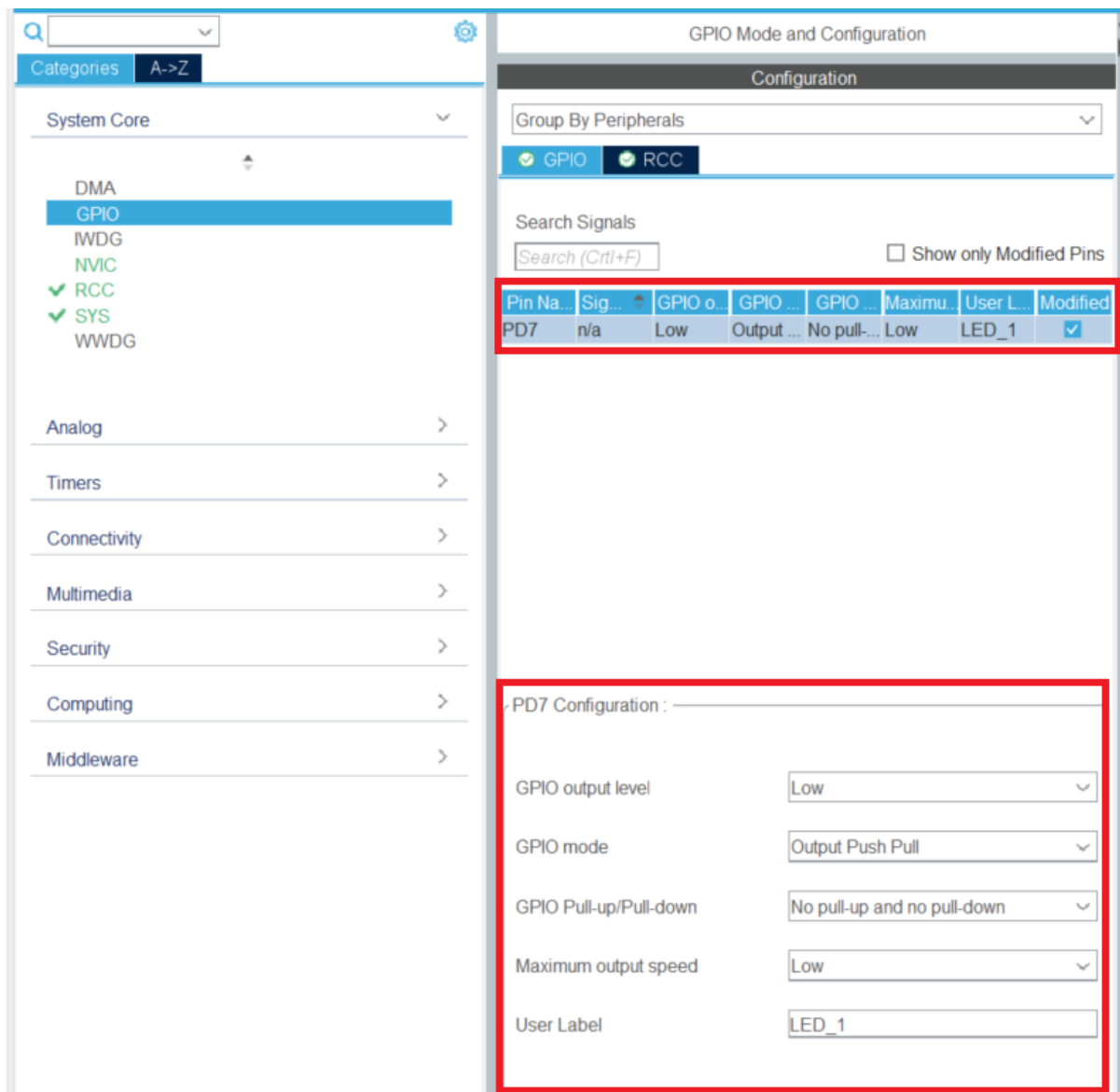
34. ábra: A megszakítás engedélyezése

- 7) A LED1 a PD7-es mikrovezérlő lábra van kötve, ezért ennek a konfigurálását is el kell végeznünk. Ezt legegyszerűbben úgy tehetjük meg, hogy a mikrovezérlő ábráján rákattintunk a PD7-es lábra és kiválasztjuk a GPIO Output-ot, hiszen a LED vezérléséhez kimenetként kell használnunk ezt a lábat.



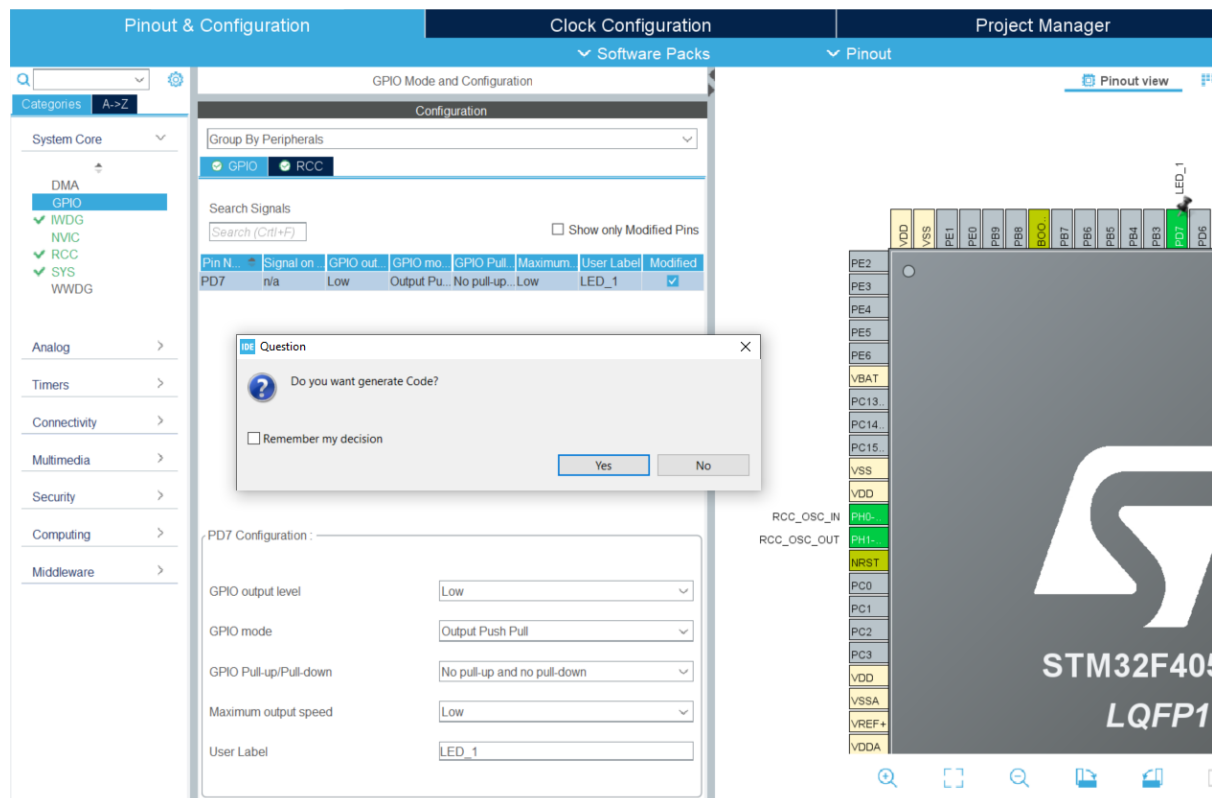
35. ábra: A LED1-et vezérlő kimenet beállítása

- 8) Térjünk vissza a System Core menühöz és kattintsunk a GPIO-ra a további beállítások elvégzéséhez. Lehetőségünk van alapértelmezett logikai jelszint kiválasztására. Esetünkben a Low, azaz alacsony jelszint megfelelő, mert inicializáláskor logikai alacsony jelszint jelenik meg a lábon, ami megfelel a lekapcsolt LED állapotnak. A User Label-nél érdemes saját elnevezést megadni, mert így a generált forráskódban áttekinthetően és az általunk preferált elnevezésnek megfelelően kezelhetjük a perifériákat. Ennek megfelelően nevezzük el a GPIO lábat például LED_1-nek.



36. ábra: Alapértelmezett jelszint kiválasztása a LED1 számára

- 9) Mentsük el a módosításokat. Ezt megtehetjük az eszköztárban a floppy lemezt szimbolizáló Save gombra kattintva. Ezt követően generáltassuk le a kódunkat.



37. ábra: A forráskód generálása

- 10) A továbbiakban a main.c fájlban fogunk dolgozni, itt implementáljuk a LED vezérlését. Először is hozzunk létre egy változót, `is_timeout_occurred` néven. A későbbiekben ezt egy úgynevezett flag-ként (esemény bekövetkezését jelölő változó) használjuk majd. Ehhez a változóhoz `uint32_t` típust választottam, ugyanis a tananyagban használt mikrovezérlő 32 bites, így ezen a bitszámon képes a leggyorsabban műveleteket végezni. Ráadásul a változó értékének átírása atomi művelet lesz ebben az esetben, ami azt jelenti, hogy semmi sem tudja megszakítani ezt a folyamatot.

```
/* Private variables -----*/
TIM_HandleTypeDef htim1;

/* USER CODE BEGIN PV */
// változó a timer lejáráásának figyelésére
static volatile uint32_t is_timeout_occurred;
```

- 11) A végtelen ciklus előtt még el kell végeznünk néhány beállítást. Az `is_timeout_occurred` változónkat inicializáljuk 0 kezdőértékkel, ez jelenti azt, hogy még nem járt le az időzítésünk. Töröljük a Timer 1 Update interrupt flag-et a számláló és időzítő modulban. Ezt azért célszerű megtenni, hogy garantáltan tiszta lappal induljon a rendszer és az első megszakításkezelő rutin

ténylegesen az első időzítés lejártá után fusson le.
Végül pedig elindítjuk a Timer 1 modult.

```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_TIM1_Init();
/* USER CODE BEGIN 2 */
// kezdeti állapot beállítása
is_timeout_occurred = 0;

// Update Interrupt flag törlése az elindítás előtt
__HAL_TIM_CLEAR_FLAG(&htim1, TIM_FLAG_UPDATE);

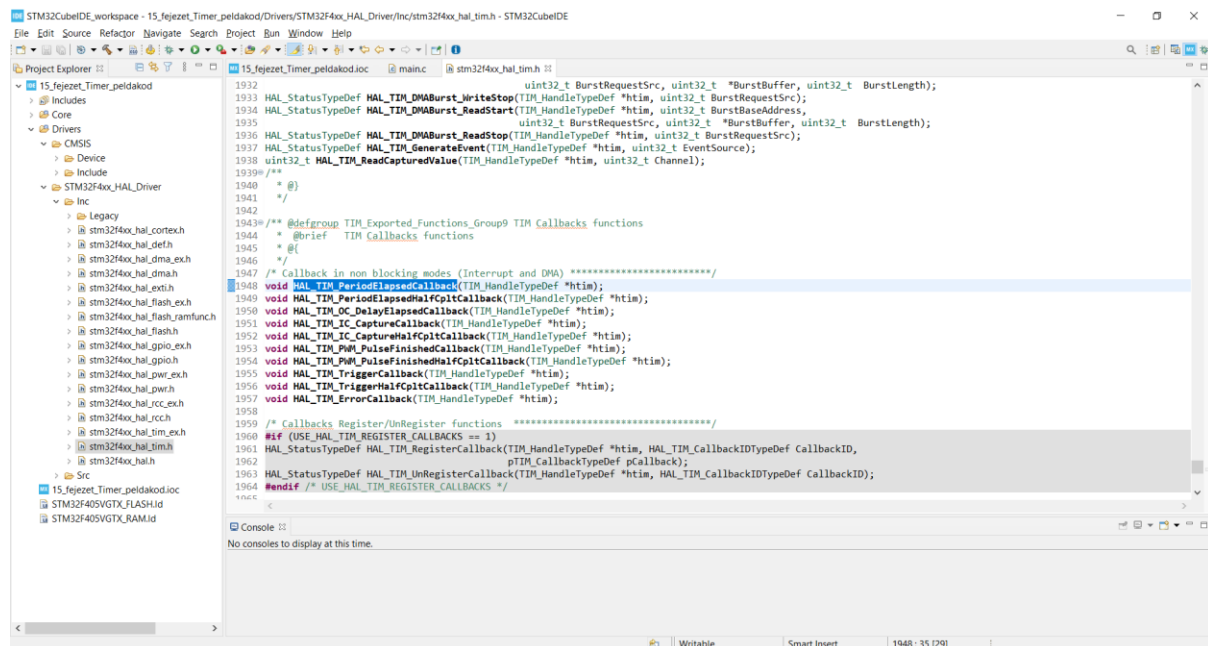
// Timer 1 elindítása
HAL_TIM_Base_Start_IT(&htim1);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

12) Következő lépésként implementáljuk a megszakításkezelő rutint. A forráskód generálásával megtörtént a `HAL_TIM_PeriodElapsedCallback()` függvény deklarációja, amit az `stm32f4xx_hal_tim.h` fájlban találunk meg. Ezt a callback függvényt hívja meg a TIM1 modul interrupt-ja, ebben tudjuk kezelni a megszakítást, ám ehhez a függvény definíciójáról is nekünk kell gondoskodnunk.



38. ábra: A forráskód generálásával megtörtént a HAL_TIM_PeriodElapsedCallback() függvény deklarációja

Definiáljuk a függvényt a main.c fájlban és állítsuk benne 1-re az is_timeout_occurred változót, ezzel fogjuk jelezni, hogy letelt a beállított időzítés.

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    // amennyiben a timer lejárt, a változó értékét 1-be állítjuk
    is_timeout_occurred = 1;
}
/* USER CODE END 4 */
```

- 13) Végül pedig programozzuk le a végtelen ciklus tartalmát. Az inicializálás után ezt a programkódot futtatja le a mikrovezérlő újra és újra, egészen a tápfeszültség megszűnéséig. A ciklus elején megvizsgáljuk, hogy letelt-e az időzítés. Emlékezzünk vissza, ezt a flag-et a TIM1 modul megszakításkezelő rutinja állítja 1-re és amint ez megtörténik, az if feltétel teljesülni fog és lefut a benne implementált kódrész. Ilyenkor gondoskodunk a flag alapállapotba állításáról, hogy legközelebb csak akkor avatkozunk be, amikor újra letelt a beállított időzítésünk. Majd pedig az ellenkezőjére változtatjuk a LED1 állapotát.

```
// Update Interrupt flag törlése az elindítás előtt
__HAL_TIM_CLEAR_FLAG(&htim1, TIM_FLAG_UPDATE);
```

```
// Timer 1 elindítása
HAL_TIM_Base_Start_IT(&htim1);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    // esemény amennyiben lejárt a timer
    if ( is_timeout_occurred )
    {
        is_timeout_occurred = 0;

        // LED1 állapotának az ellenkezőjére állítása
        HAL_GPIO_TogglePin(LED_1_GPIO_Port, LED_1_Pin);
    }
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

WATCHDOG TIMER PÉLDA

Ebben a példában is a fejlesztőpanelen található LED1-et fogjuk villogtatni úgy, hogy minden egyes LED kikapcsoláskor a Watchdog Timert is újratöltjük. Azonban az 5. LED lekapcsolást követően megakasztjuk a program futását, ezáltal abbahagyjuk a Watchdog periodikus újratöltését is. Azt fogjuk tapasztalni, hogy a Watchdog időzítő lejárása után újraindul a program, amit az ismét villogó LED1-en veszünk majd észre. A továbbiakban lépésről lépésre ismertetem a példakód működését és a szükséges beállításokat.

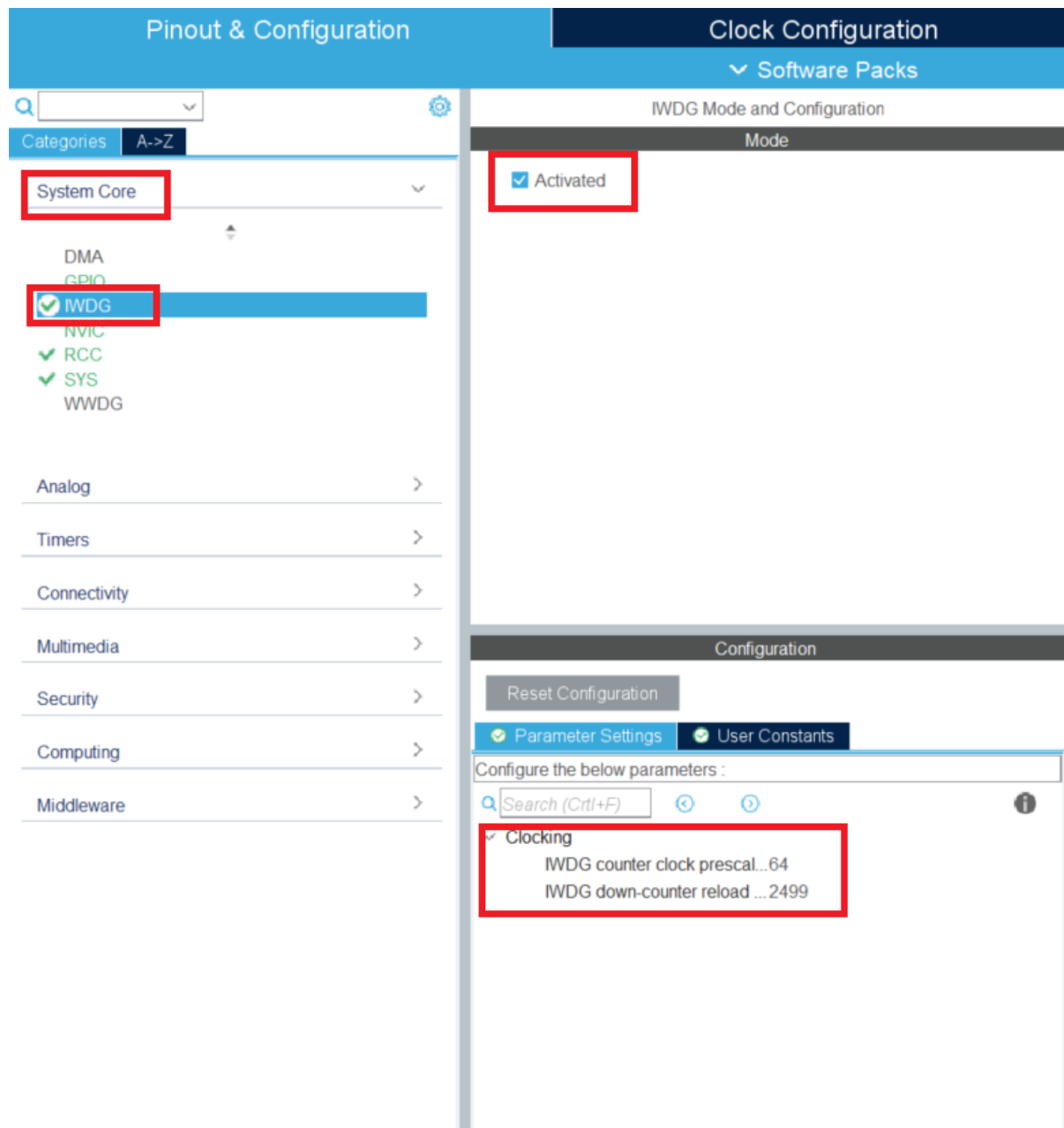
- 1) Hozzunk létre egy új projektet az STM32CubeIDE fejlesztői környezetben. Válasszuk ki az STM32F405VG mikrokontrollert és állítsuk át az órajel forrását külsőre. Ezeket a lépéseket teljesen lefedik a Timer interrupt példa 1-4 pontjai.
- 2) Állítsuk kimenetre a mikrokontroller LED1-et vezérlő lábát (PD7) a Timer interrupt példa 7. és 8. pontjainak megfelelően.

- 3) A System Core menüben kattintsunk az IWDG-re (Independent Watchdog) a Watchdog beállításaihoz. Az időzítést 5 másodpercre szeretnénk állítani, ehhez pipáljuk ki az Activated-et a modul engedélyezéséhez és állítsuk át az előosztást (IWDG counter clock prescaler) 64-re, a számlálási határt (IWDG down-counter reload value) pedig 2 499-re. Fontos megjegyezni, hogy a Watchdog periféria egy 32 kHz frekvenciájú független órajelről működik, erről a kézikönyv 709. oldalán találunk információt. Az időzítés hosszát kiszámolhatjuk a következő összefüggés alapján.

$$T [s] = \frac{Előosztás \times Limit}{f_{periféria} [Hz]} = \frac{64 \times (2\,499 + 1)}{32\,000 [Hz]} = 5 [s].$$

A Watchdog esetében kötöttek a lehetőségeink az előosztást illetően, amelyeket szintén a kézikönyv előbb említett oldalán találjuk. Ennél a perifériánál nem kell ügyelnünk rá, hogy 1-gyel kevesebb értéket állítsunk be az előosztás esetén, a számlálási limitnél azonban figyelniük kell erre.





39. ábra: Előosztás beállítása

- 4) Mentsük el a módosításokat és generáltassuk le a kódunkat.
- 5) A main.c fájlban hozzunk létre a counter és wdt_reloads változókat. Az előbbi időzítésre fogjuk használni, utóbbival pedig a Watchdog újratöltéseit tartjuk majd számon.

```

/* Private variables -----*/
IWDG_HandleTypeDef hiwdg;

/* USER CODE BEGIN PV */
static uint32_t counter;

```

```
static uint32_t wdt_reloads;  
/* USER CODE END PV */
```

6) A végtelen ciklus előtt inicializáljuk a változókat 0 kezdőértékkel.

```
/* Initialize all configured peripherals */  
MX_GPIO_Init();  
MX_IWDG_Init();  
/* USER CODE BEGIN 2 */  
// Változó az időzítéshez illetve a watchdog újratöltések számolásához  
counter = 0;  
wdt_reloads = 0;  
  
/* USER CODE END 2 */  
  
/* Infinite loop */  
/* USER CODE BEGIN WHILE */  
while (1)  
{  
    /* USER CODE END WHILE */  
  
    /* USER CODE BEGIN 3 */  
}  
/* USER CODE END 3 */
```

7) Végül pedig töltsük ki a végtelen ciklust. A Counter számláló inkrementálásával számoltassunk el 500 000-ig, ez egy szemmel is jól érzékelhető várakozást jelent, nagyjából 1 másodpercet. Amint ezt elértük belépünk az if feltételbe és felkapcsoljuk a LED1-et, majd újabb időzítés következik, ezúttal lefelé számolunk 0-ig. Ezt követően lekapcsoljuk a LED1-et, újratöltjük a Watchdog-ot, majd növeljük wdt_reloads változó értékét. Mindez ötször fog megtörténni, utána végtelen ciklusba ugrik a program, aminek köszönhetően a Watchdog-ot sem tölti újra, hagyja azt lejárni. A végtelen ciklusba lépés előtt közvetlenül elvégeztük a Watchdog újratöltését, így 5 másodperc után lejár a Watchdog időzítő és a periféria újraindulást generál, ezáltal az elejéről indul a programunk futása is, amit a villogó LED-ről veszünk észre.

```
/* USER CODE BEGIN WHILE */
while (1)
{
    // Időzítés kivárása
    counter++;
    if ( counter == 500000 )
    {
        // LED 1 felkapcsolása
        HAL_GPIO_WritePin(LED_1_GPIO_Port, LED_1_Pin, GPIO_PIN_SET);

        // Időzítés kivárása
        while(counter > 0)
        {
            counter--;
        }

        // LED 1 lekapcsolása
        HAL_GPIO_WritePin(LED_1_GPIO_Port, LED_1_Pin, GPIO_PIN_RESET);

        // Watchdog újratöltése
        HAL_IWDG_Refresh(&hiwdg);
        wdt_reloads++;

        // Program szándékos megakasztása 5 újratöltést követően
        if ( wdt_reloads == 5 )
        {
            while(1);
        }
    }
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

ÖSSZEFOGLALÓ

A fejezet során megismerkedtünk a számláló és időzítő modulok leggyakoribb üzemmódjaival és beállításaival.

Szó esett az egyszerű időzítő funkcióról, a bemeneteken történő események detektálásáról (Input capture), az időzített kimenet állításról (Output compare), beleértve az impulzusszélesség-modulációt (PWM). Megvizsgáltuk, hogy miként lehet a forgó mozgások jellemzőit figyelemmel kísérni egy mikrokontroller segítségével (Enkóder interfész), majd rátértünk a beágyazott rendszerek védelmére szolgáló Watchdog perifériára. Röviden kitértünk a SysTick Timer-re is, amely a beágyazott operációs rendszerek számára biztosít pontos időalapot.

Gyakorlati példák segítségével pedig LED-et villogtattunk pontos időzítéssel, valamint láthattuk, hogy a Watchdog miként képes megvédeni a szoftvert a megakadásától.